Journal of *Cybersecurity and Privacy*

MDPI

Article

An Explainable Hybrid Pipeline for Malware Classification: Benchmark Construction, Feature Reduction, and Security-Oriented Evaluation

Carmelo Ardito [1,*], Giuseppe Loseto [2], Riccardo Di Pietro [3], Nicola Epicoco [2] and Alessandro Massaro [2]

[1] Department of Electrical and Information Engineering, Polytechnic University of Bari, 70125 Bari, Italy
[2] Department of Engineering, LUM "Giuseppe Degennaro" University, 70010 Casamassima, Italy; loseto@lum.it (G.L.); epicoco@lum.it (N.E.); massaro@lum.it (A.M.)
[3] Deloitte Consulting, 00144 Rome, Italy
* Correspondence: carmelo.ardito@poliba.it

Abstract

Malware classification increasingly relies on machine learning models that combine static and dynamic evidence, yet their practical use is often limited by dataset inconsistency, high-dimensional feature spaces, and insufficient transparency. This paper presents an explainable hybrid malware-classification pipeline built on an aligned public dataset in which static and dynamic features are matched at sample level and share the same class space. The framework combines a Random Forest static branch, a calibrated XGBoost dynamic branch, and a weighted late-fusion stage whose branch weights are derived from inner-validation weighted-F1 rather than from test performance. On the corrected no-leak benchmark, static reduction compresses the static space from 771 to 258 features, while sparse-aggressive reduction compresses the dynamic space from 21,918 to 374 features. An early-fusion XGBoost baseline achieves the best multiclass aggregate scores, whereas the validation-weighted calibrated hybrid provides the strongest false-negative-first `Benign` vs. `Malware` profile, reaching malware recall 0.9998, benign recall 0.8053, and one false negative on the test set. The study shows that, once leakage is removed and fusion is validation-driven, the preferred hybrid architecture depends on the operational objective rather than on a single aggregate metric.

Keywords: malware classification; explainable AI; hybrid malware analysis; feature reduction; security-oriented evaluation

Check for updates

Academic Editor: Georgios Kambourakis

Received: 17 April 2026
Revised: 22 May 2026
Accepted: 8 June 2026
Published: 22 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Malware continues to evolve in volume, diversity, and sophistication, creating sustained pressure on industrial [1] and enterprise security operations [2]. Signature-based approaches remain useful in practice, but they are not sufficient against rapidly changing malware families, packed executables, or previously unseen variants. For this reason, Machine Learning (ML) has become a central technique in malware analysis, both for binary detection and for family-level classification [3,4].

Among ML-based approaches, hybrid pipelines that combine static and dynamic evidence are particularly attractive. Static analysis is efficient and safe because it inspects the executable without running it, but it can be weakened by packing, obfuscation, or sparse behavioral semantics. Dynamic analysis captures behavioral traces and Application Programming Interface (API) usage at execution time, but it is more expensive and may be

affected by environment-aware malware [5,6]. A modular hybrid architecture can, therefore, exploit the complementary strengths of both branches.

However, building a methodologically sound hybrid malware-classification workflow is not just a matter of combining existing libraries. Three issues remain especially important. First, public datasets are often inconsistent with the intended architecture: static and dynamic partitions may not be aligned at sample level, labels may differ across modalities, or the available feature spaces may support only partial tasks. Second, dynamic feature blocks are frequently sparse and high-dimensional, which makes feature reduction and model selection crucial for tractable training and meaningful comparison [7–9]. Third, explainability is often treated as a visualization add-on rather than as an auditing layer tied to model design and evaluation.

This work addresses these issues through a literature-informed hybrid pipeline that is explicitly built on an aligned public dataset. The contribution is not a radically new learning algorithm. Rather, the paper proposes a coherent end-to-end design that connects dataset selection, modality-specific preprocessing, validation-aware dynamic optimization, decision fusion, and explainability under a common experimental protocol. The novelty of the study is, therefore, methodological rather than algorithmic: it lies in the construction of a leakage-aware aligned benchmark, in the explicit optimization of heterogeneous static and dynamic branches, in validation-driven decision fusion, and in security-oriented evaluation under a false-negative-sensitive criterion.

The paper is guided by the following research questions:

- RQ1: Which publicly available dataset is the most methodologically suitable for building a coherent hybrid static/dynamic malware-classification pipeline?
- RQ2: How do branch-specific feature-reduction and dynamic-branch optimization strategies affect predictive performance, dimensionality, and computational cost in an aligned hybrid malware benchmark?
- RQ3: Compared with reproducible fusion baselines, which final hybrid configuration provides the best trade-off between multiclass performance and security-oriented `Benign` vs. `Malware` evaluation?

The ***main contributions*** of the paper are as follows:

- A literature-informed hybrid malware-classification pipeline built on an aligned public benchmark and organized as a static branch, dynamic branch, decision fusion, and eXplainable Artificial Intelligence (XAI) audit layer;
- A methodological dataset-selection process and benchmark-construction procedure designed to support coherent hybrid learning across static and dynamic modalities;
- A systematic experimental analysis of branch-specific reduction, dynamic-branch optimization, and decision-fusion strategies against reproducible baselines;
- An evaluation framework that combines multiclass metrics with security-oriented `Benign` vs. `Malware` criteria to guide the final-system choice.

The remainder of the paper is organized as follows. Section 2 summarizes the literature review and the design requirements derived from it. Section 3 discusses candidate datasets and motivates the choice of the final benchmark. Section 4 presents the architecture and the preprocessing pipeline. Section 5 reports the experimental protocol and the corrected results. Section 6 discusses the main implications of the findings. Section 7 states the main limitations of the study. Section 8 concludes the paper.

2. Related Work and Design Requirements

The review basis of this work is a structured literature analysis rather than an ad hoc model assembly. An initial systematic search was conducted on ACM Digital Library and

IEEE Xplore using title-based queries targeting ML/AI models and techniques for malware analysis. The search returned 110 papers, which were independently screened by three reviewers with academic and industrial backgrounds. After applying reconciliation and explicit inclusion/exclusion criteria, 14 studies published between 2019 and 2025 were retained as the core review corpus. For transparency, Appendix A reports the complete list of screened studies, while Appendix B reports the detailed analysis table of the selected papers. Following the systematic review, an additional targeted search was conducted to capture more recent studies on the themes that emerged most clearly from the review, especially explainability-oriented trends in malware analysis and cybersecurity. Eight candidate papers were identified, and five were retained for contextual discussion [10–14].

This review process matters for two reasons. First, it provides a principled basis for the design decisions of the pipeline. Second, it helps delimit what the present work should and should not claim. The resulting paper is positioned as a methodologically coherent, experimentally grounded hybrid pipeline rather than as a claim of algorithmic novelty or state-of-the-art superiority.

2.1. Main Trends

The main trends emerging from the core review corpus confirm the strong role of supervised learning in malware analysis. Ensemble methods such as Random Forest and gradient-boosted trees remain widely used because they combine robust performance with manageable interpretability [5,7,15,16]. Deep learning models are also important, especially for image-based malware representations and sequential traces, but their opacity and higher operational cost remain problematic in security-critical contexts [6,17,18]. The targeted update broadly reinforces this picture. Recent explainability-oriented malware studies continue to combine high-performing supervised classifiers with post hoc tools such as SHAP and LIME, typically as auditing add-ons rather than as central architectural principles [10]. At a broader level, recent surveys on XAI in cybersecurity stress that explainability work remains fragmented across tasks, techniques, and evaluation goals [11,12].

Another consistent trend concerns feature engineering. Recent studies show that careful reduction in high-dimensional feature spaces can improve not only computational efficiency but also classification quality and interpretability [7–9]. This is especially relevant for dynamic features such as API traces and behavioral indicators, which are often sparse, redundant, and expensive to process. Recent broad reviews of AI/ML in cybersecurity also place transparency, adversarial robustness, and data realism among the main unresolved requirements for operational adoption, which is consistent with the concerns emerging from the malware-specific literature reviewed here [13].

2.2. Main Gaps

The main gaps identified by the review concern the current literature on hybrid malware-analysis pipelines. These pipelines are repeatedly presented as a way to combine the efficiency of static analysis with the richer semantics of dynamic behavior [5,6,19,20]. At the same time, the literature reveals several recurring methodological weaknesses:

- First, many publicly available malware datasets remain difficult to reuse in a coherent hybrid setting because static and dynamic partitions are often not directly comparable or are poorly aligned with the intended shared task. This complicates fair fusion between modalities and may force artificial relabeling or mismatched output spaces.
- Second, many papers emphasize aggregate multiclass metrics without making explicit the trade-off between these scores and operationally critical errors, especially false negatives in malware-screening settings where missing a malicious sample is more consequential than generating an additional alert.

- Third, XAI is frequently used as a descriptive visualization layer rather than as a tool for model auditing, debugging, and structured inspection, and the broader XAI-for-cybersecurity literature still lacks unified comparative frameworks and strong evaluation practices [11,12]. More recent contextual work confirms that this area is evolving but still highly task-specific: for example, ANAKIN combines competitive malware detection with both global and local explanations, including the analysis of missed alerts, but does so in the specific setting of Android API-call graphs rather than in an aligned Windows static/dynamic family-classification benchmark [14]. This reinforces the value of explainability, while also showing that many recent advances are not directly transferable to the hybrid setting addressed in the present paper.

2.3. Design Requirements

These findings led to five practical design requirements for the present work:

- The final benchmark must align static and dynamic evidence at sample level and support a coherent shared class space;
- The static and dynamic branches must be optimized separately because their feature spaces have different statistical properties;
- Dynamic preprocessing must explicitly address sparse, high-dimensional inputs;
- The decision-fusion block must be compared against reproducible baselines, not only against the individual branches;
- Explainability must be integrated as an XAI audit layer, with explicit caution against causal interpretation.

Taken together, these requirements also clarify the intended contribution of the paper with respect to prior work. The goal is not to claim that hybrid malware classification is new, nor that explainability alone makes the resulting system more trustworthy. Rather, the contribution lies in making several methodological choices explicit and empirically traceable: selecting a dataset that genuinely supports hybrid learning, separating the optimization logic of the two branches, comparing fusion strategies under a shared benchmark, and reporting results under both aggregate multiclass and security-oriented criteria. This framing is important because many weaknesses in the literature do not arise from the choice of classifier alone but from mismatches between data, architecture, and evaluation protocol. These requirements directly inform the dataset selection, the feature-reduction strategy, the choice of branch-level models, and the evaluation protocol described in the next sections.

3. Dataset Selection and Benchmark Construction

Because the proposed framework depends on supervised learning and on the coherent interaction between static and dynamic modalities, dataset selection is a central methodological step. Table 1 summarizes the public datasets considered during the screening phase.

The comparison was guided by four explicit criteria:

- **Sample-level alignment**: static (S) and dynamic (D) data should be linkable for the same executable;
- **Shared target space**: the two modalities should support a coherent common label set without forced relabeling;
- **Modality completeness**: the benchmark should expose both static and dynamic evidence usable in the same study;
- **Practical reusability**: the dataset should be publicly usable without extensive external enrichment or undocumented reconstruction steps.

Table 1. Candidate datasets considered during screening. The final decision was not based only on preliminary promise, but on suitability for a coherent hybrid multiclass pipeline.

ID	Dataset	Modalities	Main Issue for This Study	Decision
1	Multi-feature Dataset for Windows PE Malware Classification [21]	S + D	Low preliminary suitability and limited scale for the intended multi-family benchmark	Rejected
2	Windows PE-Malware API Dataset [22]	S + D	Strong missing-value burden on static side and dependence on external enrichment for practical reuse	Rejected
3	Malware Classification and Labeling using Deep Neural Networks [23]	D only	Dynamic-only dataset; unsuitable for a coherent two-branch hybrid design	Rejected
4	Malware images: visualization and automatic classification [24]	Image	Visual representation only; excluded from the target architecture	Rejected
5	Malware Detection using Machine Learning (MDML) [25]	S + D	Static and dynamic partitions are not coherently aligned for the intended shared class-space pipeline	Rejected
6	API Sequences Malware Datasets [26]	D only	API-sequence focus only; incomplete for hybrid static/dynamic fusion	Rejected
7	Blue Hexagon Open Dataset for Malware AnalysiS (BODMAS) [27]	S only	Static-oriented benchmark; useful as external reference but not as final hybrid dataset	Rejected
8	Malware static and dynamic features VxHeaven and VirusTotal [28]	S + D	Binary emphasis and limited clarity for the intended aligned multiclass benchmark	Rejected
9	Malware Open-Source Threat Intelligence Family dataset [29]	S only	Rich family granularity but no matching dynamic block	Rejected
10	Quo Vadis dynamic malware dataset [19]	D + context	Valuable related benchmark, but not a directly reusable aligned static/dynamic public dataset	Rejected
11	Windows Malwares dataset [30]	S + D	Shared sample identifiers and coherent family-level integration across raw CSV files	Selected

Within this comparison, MDML deserves explicit clarification because it initially appeared promising in the early screening phase. It contains both static and dynamic malware-related information and showed encouraging preliminary performance. However, it was ultimately discarded for methodological reasons. Its static and dynamic partitions were not coherently aligned for the intended unified multiclass framework: the two branches did not naturally expose the same target structure, and a shared class space would have required non-natural aggregation decisions. Using MDML as the final benchmark would, therefore, have forced asymmetric branch outputs and an artificial fusion logic dictated by the dataset rather than by the intended architecture. This point matters because it illustrates a broader lesson of hybrid malware-classification research: a dataset may look attractive in isolation and still be unsuitable for a coherent end-to-end design. In this study,

MDML is, therefore, treated only as a candidate considered during dataset selection, not as the dataset adopted for the final system.

The final benchmark is built from the Windows Malwares dataset [30], which exposes raw static and dynamic sources that can be aligned by sample through `sha256`. The four raw files used are as follows, with each pair denoting samples and features:

- Portable Executable Header—`PE_Header.csv`: $(29{,}807 \times 54)$;
- Portable Executable Section—`PE_Section.csv`: $(29{,}760 \times 92)$;
- Dynamic Link Libraries—`DLLs_Imported.csv`: $(29{,}498 \times 631)$;
- Application Programming Interface—`API_Functions.csv`: $(29{,}505 \times 21{,}920)$.

The first three files constitute the static block, while `API_Functions.csv` forms the dynamic block. After merging the static files and intersecting the shared `sha256` values with the dynamic file, the final aligned benchmark contains 29,489 samples and 22,694 columns overall. The class space consists of seven labels: `Benign`, `RedLineStealer`, `Downloader`, `RAT`, `BankingTrojan`, `SnakeKeyLogger`, and `Spyware`. This aligned benchmark is central to the methodology because it allows both branches to learn on the same sample universe and the same target space, which, in turn, makes probability-level fusion theoretically consistent.

4. Materials and Methods

4.1. Pipeline Overview

The pipeline is organized into five phases:

1. Aligned dataset construction from raw static and dynamic sources;
2. Preprocessing and feature reduction, performed separately for the static and dynamic blocks;
3. Branch-level training for the static and dynamic classifiers;
4. Decision fusion through weighted averaging of class probabilities;
5. Post hoc auditing through the XAI audit layer.

Figure 1 summarizes the full workflow. The key architectural point is that the system does not start directly from two independent feature blocks sent to two models. Instead, raw static and dynamic sources are first aligned at sample level through `sha256`, and only then passed to branch-specific preprocessing and reduction pipelines. This ensures that the static branch, the dynamic branch, and the decision-fusion block all operate on a coherent benchmark.

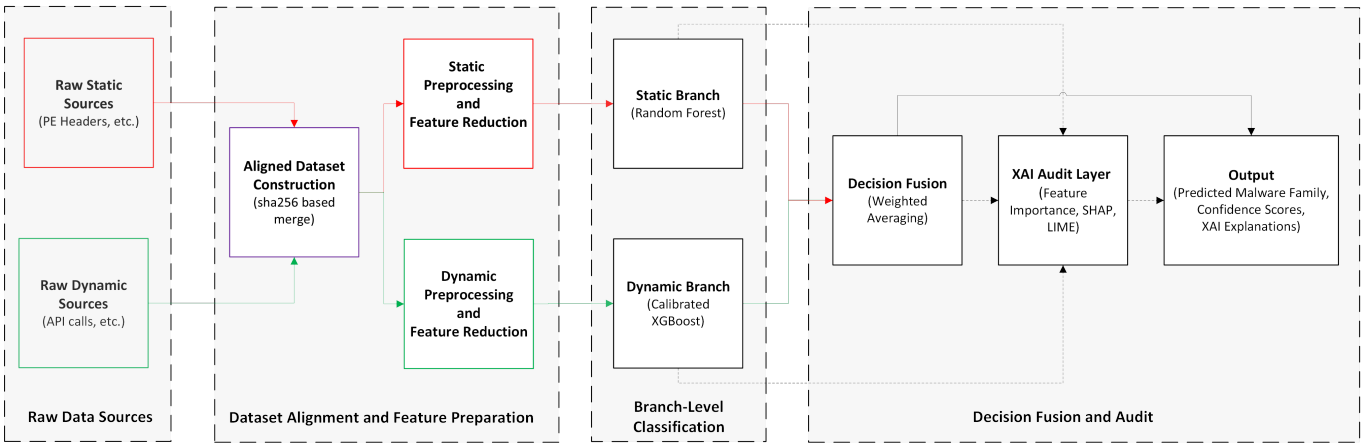


Figure 1. Architecture of the proposed pipeline. Raw static and dynamic sources are first aligned at sample level, then processed through branch-specific preprocessing and feature-reduction stages, followed by branch-level classification, weighted decision fusion, and the XAI audit layer. Red arrows denote the static-analysis flow, green arrows denote the dynamic-analysis flow.

The global benchmark uses a stratified hold-out split with 80% training data and 20% testing data, with fixed random seed. An internal stratified validation split is then extracted only from the training portion. For the dynamic branch, this inner split supports candidate selection, class-aware calibration, and specialist-model tuning. For the static branch, it is used only to estimate the branch-level validation score later used in weighted decision fusion. In all cases, the test set remains untouched until final evaluation.

4.2. Branch Preparation, Modeling, and Decision Fusion

Before model training, all label-like fields are removed from the feature matrix to prevent leakage. This includes columns whose names directly encode target semantics. The resulting feature spaces before reduction are as follows:

- Static block: $(29{,}489 \times 771)$;
- Dynamic block: $(29{,}489 \times 21{,}918)$.

The static branch uses a standard reduction pipeline composed of three steps:

1. Variance threshold: a simple feature selection technique that removes all features whose variance falls below a specified threshold, helping eliminate low-information or constant features from a dataset;
2. Correlation filtering: removing highly correlated variables from a dataset to reduce redundancy and improve model performance by keeping only the most informative and independent features;
3. `SelectFromModel(RandomForestClassifier, threshold="median")` procedure: selecting features whose importance scores, as determined by a Random Forest model, are above the median importance value, effectively retaining only the most relevant features for the classification task.

This reduces the static block from 771 to 258 features. The rationale behind this sequence is pragmatic. Low-variance and highly correlated static variables contribute little additional information while increasing model complexity and the risk of redundant importance patterns. The final model-based filter retains variables that remain useful for tree-based discrimination on the training set, yielding a compact static representation that stays compatible with global feature-importance analysis.

The dynamic branch uses a more specialized reduction strategy for sparse high-dimensional data:

1. Minimum-presence filtering: removing features or variables that appear in too few samples, ensuring that only those with sufficient representation across the dataset are retained for analysis;
2. `SelectKBest(chi2)`: ranking features based on the chi-squared statistical test and selecting the top k features that have the strongest relationship with the target variable;
3. `SelectFromModel(ExtraTreesClassifier)`: using the feature-importance scores computed by an ExtraTrees model to automatically retain only the most relevant features.

This *sparse-aggressive* strategy reduces the dynamic block from 21,918 to 374 features. This reduction pipeline reflects the very different statistical profile of the dynamic block. The initial-presence filter removes API indicators that are too rare to support stable estimation, the chi-square stage preserves variables with stronger class association, and the final ExtraTrees-based selector further removes redundancy under a non-linear criterion. The goal is not only to compress the feature space, but also to make the dynamic branch trainable and auditable without collapsing its discriminatory signal.

At branch level, the static classifier is trained with `RandomForestClassifier`. This choice is consistent with the literature, which frequently reports strong performance and manageable interpretability for tree-based ensembles on static malware features [7,15,16]. As shown in Figure 1, the static classifier operates only after the aligned dataset has been filtered through the static preprocessing and reduction pipeline. The static branch is, therefore, optimized through the reduced feature space rather than through complex branch-specific post-processing. To keep the later weighting step free from test information, an auxiliary static fit is also evaluated on the internal validation subset, and only this validation weighted-F1 is exported as the static-branch weighting score. The dynamic branch, by contrast, starts from the sparse-aggressive reduced representation and then applies validation-aware optimization. Two candidate models are compared on the inner-validation split:

- `XGBClassifier`;
- `LogisticRegression` with `saga` solver.

The inner-validation subset corresponds to 20% of the training partition and is stratified by class. Class weights are used to mitigate imbalance effects at training time. In addition, the dynamic branch applies class-aware probability calibration by learning multiplicative adjustments on validation data from the grid $\{0.75, 1.0, 1.25\}$. In the final corrected benchmark, the selected configuration is an `XGBClassifier` with histogram-based tree construction (`tree_method = hist`), while the learned calibration multipliers are {1.0, 0.75, 1.25, 0.75, 1.0, 1.0, and 1.25} for classes 0–6, respectively. Two specialist extensions were also explored on the confused subset {3, 4, 5, and 6}, but the calibrated branch remained the strongest dynamic configuration overall. In the architecture Figure, this branch is, therefore, represented as a calibrated dynamic module rather than as a generic dynamic classifier.

The final decision-fusion step combines static and dynamic class probabilities through weighted averaging. In the corrected benchmark, both branches operate on the same seven-class output space. Let $p_s(c \mid x)$ and $p_d(c \mid x)$ denote the static and dynamic probabilities for class c and sample x. The final score is

$$\hat{p}(c \mid x) = w_s p_s(c \mid x) + w_d p_d(c \mid x), \tag{1}$$

where the weights are derived from validation-weighted-F1 scores estimated only on the inner-validation subsets of the two branches:

$$w_s = \frac{WF1_s}{WF1_s + WF1_d}, \qquad w_d = \frac{WF1_d}{WF1_s + WF1_d}. \tag{2}$$

This design is intentionally simple and reproducible, keeps the test set completely outside the weighting rule, and is directly compared in the experiments against two baselines:

- An early-fusion baseline, where the reduced static and dynamic features are concatenated and classified by a single XGBoost model;
- An equal-weight late-fusion baseline, where the two branches contribute with fixed weights 0.5/0.5.

This point also explains why the architecture diagram represents the final block as a *decision-fusion* stage based on weighted averaging, not as a separate meta-classifier. The fusion logic depends on branch probabilities and branch performance, rather than on a learned stacking model. The choice of weighted averaging also serves a methodological purpose. Because the study aims to compare architectures under a controlled benchmark, a transparent fusion rule makes it easier to attribute improvements to branch complementarity rather than to the extra flexibility of a learned meta-model. In this sense, the fusion

block acts as an interpretable decision layer whose behavior can be related back to the relative strengths of the static and dynamic branches.

4.3. XAI Audit Layer

In the proposed architecture, explainability is implemented as a post hoc audit layer for branch-level and decision-fusion inspection. Its purpose is diagnostic: to identify dominant variables in the reduced branch models, assess whether fusion behavior depends on a narrow subset of signals, and support the inspection of selected predictions. In the present study, the primary users of these explanations are researchers and engineers who audit model behavior, compare branch designs, and inspect rare errors; their value for Security Operations Center analysts is treated as secondary and limited to the inspection of selected safety-critical cases.

At branch level, the audit relies on global explanation methods that remain compatible with the adopted classifiers. For the tree-based branches, we compute impurity-based feature-importance rankings and SHAP summaries on the reduced feature spaces. These global views are used to verify whether aggressive reduction preserves a concentrated and analyzable decision basis, and to identify features whose dominance may deserve additional scrutiny. At decision level, the design also supports local inspection of selected fused outputs through LIME-style explanations. Because the main goal of the present study is to benchmark the hybrid pipeline rather than to conduct a dedicated analyst-facing explanation study, the reported XAI results emphasize global branch-level explanations and include only one illustrative local case study on the corrected benchmark.

For this reason, the XAI layer is interpreted statistically and diagnostically. It indicates which inputs are associated with the learned-decision function, but it does not establish causal relations, semantic ground truth about malware behavior, or validated human trust in the model outputs.

5. Results

5.1. Protocol and Metrics

All preprocessing stages are fit on the training data and then transferred to the test data. The training partition is further split into stratified train/validation subsets only on the training side. For the dynamic branch, this internal validation split is used to compare candidate models, learn class-aware probability multipliers, and configure specialist overlays. For the weighted late-fusion system, the same training-side logic provides the branch-validation weighted-F1 scores used to compute the fusion weights, so that the test set remains untouched until the final evaluation.

This protocol supports controlled comparison between architectural variants, but it does not constitute a temporal or out-of-distribution evaluation. For this reason, the paper reports both multiclass metrics and security-oriented binary metrics. The multiclass evaluation includes accuracy, macro-F1, and weighted-F1. The security-oriented evaluation collapses the seven classes into `Benign` vs. `Malware` and reports malware recall, malware F2, false negatives, and benign recall. The F2-score is included because it gives more weight to recall than to precision, which is appropriate when missing a malicious sample is operationally more costly than generating an additional alert.

For reproducibility, Table 2 summarizes the main classifier hyperparameters used in the corrected benchmark, while Table 3 reports the execution environment and software stack.

Table 2. Main classifier hyperparameters used in the corrected benchmark.

Model	Hyperparameters
Static Random Forest	`n_estimators=100`, `class_weight=balanced`, `random_state=42`
Dynamic XGBoost candidate	`objective=multi:softprob`, `num_class=7`, `eval_metric=mlogloss`, `tree_method=hist`, `random_state=42`
Dynamic Logistic-Regression candidate	`solver=saga`, `penalty=l2`, `class_weight=balanced`, `max_iter=2000`, `random_state=42`
Early-Fusion XGBoost	`objective=multi:softprob`, `num_class=7`, `eval_metric=mlogloss`, `tree_method=hist`, `random_state=42`

Table 3. Execution environment used for the corrected benchmark.

Component	Specification
Operating system	Microsoft Windows 11 Enterprise (`10.0.26200`)
CPU	11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00 GHz (4 cores, 8 logical processors)
RAM	31.71 GiB
Python	3.10.11
Main libraries	`scikit-learn 1.7.2`, `xgboost 3.2.0`, `pandas 2.3.3`, `numpy 2.2.6`, `matplotlib 3.10.8`, `shap 0.49.1`, `lime 0.2.0.1`, `joblib 1.5.3`

5.2. Dynamic-Branch Progression

Before the final corrected benchmark was consolidated, the dynamic branch was explored through a progression of increasingly specialized variants. Table 4 summarizes the results, while Figure 2 visualizes the compression/performance relationship across the same variants. The progression shows that sparse-aggressive reduction compresses the dynamic block from 21,918 to 374 features with only limited metric variation relative to the unreduced and standard-reduction configurations. Among the tested reduced variants, the calibrated branch attains the strongest macro-F1 and weighted-F1 values, while the two specialist extensions remain close but lower.

Table 4. Exploratory progression of the dynamic branch. These runs were useful for selecting the dynamic strategy, but the final paper reports absolute system-level results only on the corrected no-leak benchmark.

Experiment	Features	Accuracy	Macro-F1	W-F1
No reduction	21,918	0.6874	0.6434	0.6742
Standard reduction	19,726	0.6874	0.6434	0.6742
`sparse_aggressive_v1`	374	0.6858	0.6407	0.6722
`dynamic_calibrated_v1`	374	0.6824	0.6562	0.6806
`dynamic_specialist_v1`	374	0.6818	0.6552	0.6796
`dynamic_specialist_v2`	374	0.6821	0.6559	0.6803

5.3. Corrected No-Leak Benchmark

The manuscript-ready benchmark is the corrected run `dynamic_calibrated_noleak_v1`, where label-like fields are excluded from the static features. Table 5 reports the multiclass results, while Table 6 reports the security-oriented `Benign` vs. `Malware` view. Figure 3 summarizes the trade-off between weighted-F1 and malware recall across the main compared systems. Taken together, Tables 5 and 6 show that early fusion attains the highest aggregate multiclass scores, whereas the weighted calibrated hybrid attains the highest malware

recall and the lowest number of false negatives. In the corrected no-leak run, the validation-weighted-F1 values used for late-fusion weighting are 0.8879 for the static branch and 0.6692 for the dynamic branch, yielding normalized weights 0.5702 and 0.4298, respectively.

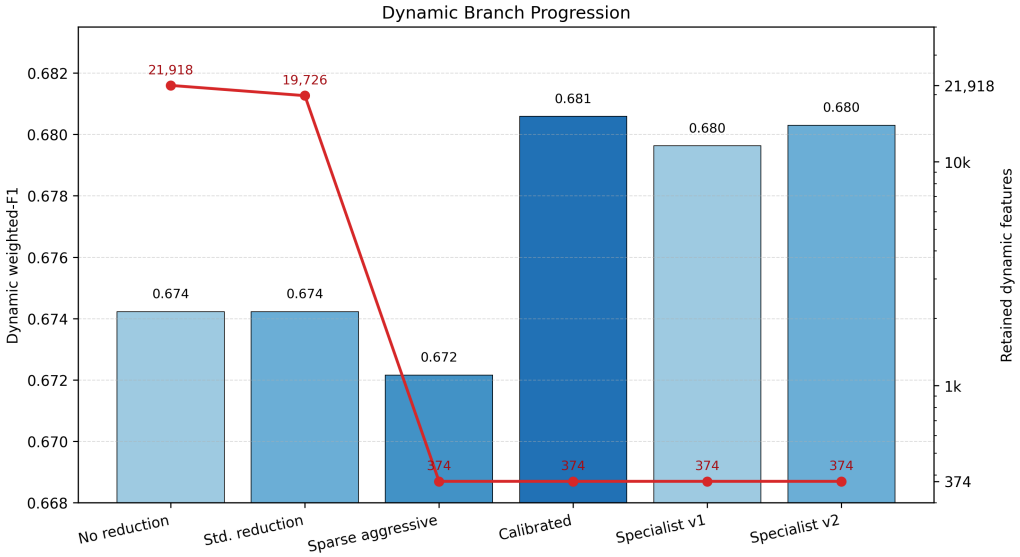


Figure 2. Dynamic branch progression across reduction and optimization strategies. The bars report weighted-F1, while the line reports the number of retained dynamic features.

Table 5. Corrected multiclass benchmark on the aligned dataset.

System	Accuracy	Macro-F1	W-F1
Static-only	0.8910	0.8922	0.8918
Dynamic-only calibrated	0.6824	0.6562	0.6806
Early-fusion XGBoost	0.8969	0.8983	0.8979
Equal-weight late fusion	0.8779	0.8714	0.8804
Weighted calibrated hybrid	0.8854	0.8825	0.8873

Table 6. Security-oriented benchmark after collapsing the task to `Benign` vs. `Malware`. `F2` emphasizes recall over precision, while `FN` denotes the number of false negatives.

System	Malware Recall	F2	FN	Benign Recall
Static-only	0.9976	0.9974	13	0.9440
Dynamic-only calibrated	0.9991	0.9914	5	0.4187
Early-fusion XGBoost	0.9978	0.9977	12	0.9600
Equal-weight late fusion	0.9996	0.9960	2	0.7280
Weighted calibrated hybrid	0.9998	0.9972	1	0.8053

Class-wise behavior further clarifies the differences between the compared systems. Table 7 reports the class-wise F1-score for the main configurations. The strongest class across all systems is `Downloader`, which remains close to perfect even for the dynamic branch alone. The most informative differences concern `Benign`, `RAT`, `SnakeKeyLogger`, and `Spyware`. The class-wise F1 values show that the dynamic branch remains strong on `Downloader` and `RedLineStealer` but drops substantially on `SnakeKeyLogger` and `Spyware`. The weighted calibrated hybrid remains much closer to the static branch than to the dynamic branch on most classes.

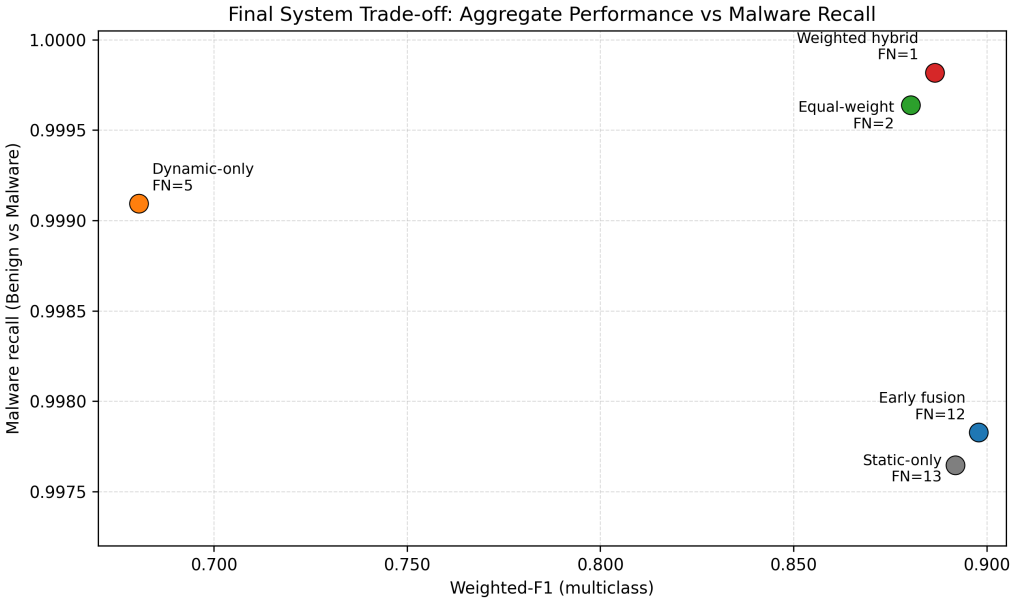


Figure 3. Trade-off between multiclass weighted-F1 and malware recall on the corrected no-leak benchmark. Labels also report the number of false negatives.

Table 7. Class-wise F1-score on the corrected benchmark.

Class	Static	Dynamic	Early-Fusion	Weighted
Benign	0.9542	0.5847	0.9639	0.8909
RedLineStealer	0.9470	0.9128	0.9514	0.9467
Downloader	0.9909	0.9898	0.9957	0.9893
RAT	0.8615	0.6037	0.8755	0.8627
BankingTrojan	0.9127	0.8230	0.9212	0.9136
SnakeKeyLogger	0.7867	0.2117	0.7877	0.7893
Spyware	0.7928	0.4675	0.7928	0.7855

To complement the aggregate metrics, Figure 4 reports the multiclass confusion matrices of the two most informative final-system configurations: early fusion, which leads the aggregate multiclass benchmark, and the weighted calibrated hybrid, which leads the false-negative-first security view.

From a computational-cost perspective, the final benchmark also highlights the asymmetry of the pipeline. Table 8 reports the execution time of the main phases in the corrected run `dynamic_calibrated_noleak_v1`. The timing results indicate that preprocessing dominates total execution time, while the weighted fusion stage adds limited overhead relative to model training.

To make the role of the XAI audit layer concrete, Figure 5 reports the global feature-importance ranking of the calibrated dynamic branch on the corrected benchmark, while Figure 6 reports the complementary SHAP summary. After sparse-aggressive reduction, both views indicate that the explanatory profile of the dynamic branch is concentrated on a limited subset of retained variables rather than being uniformly distributed across all 374 features. In the impurity-based ranking, the most influential variables include `getmessagea`, `eventsinkaddref`, `leavecriticalsection`, `vbaaryunlock`, and `fmode`. The SHAP summary confirms a similarly concentrated profile, with overlapping emphasis on low-level library, runtime, and memory-management calls rather than on directly human-readable malware actions.

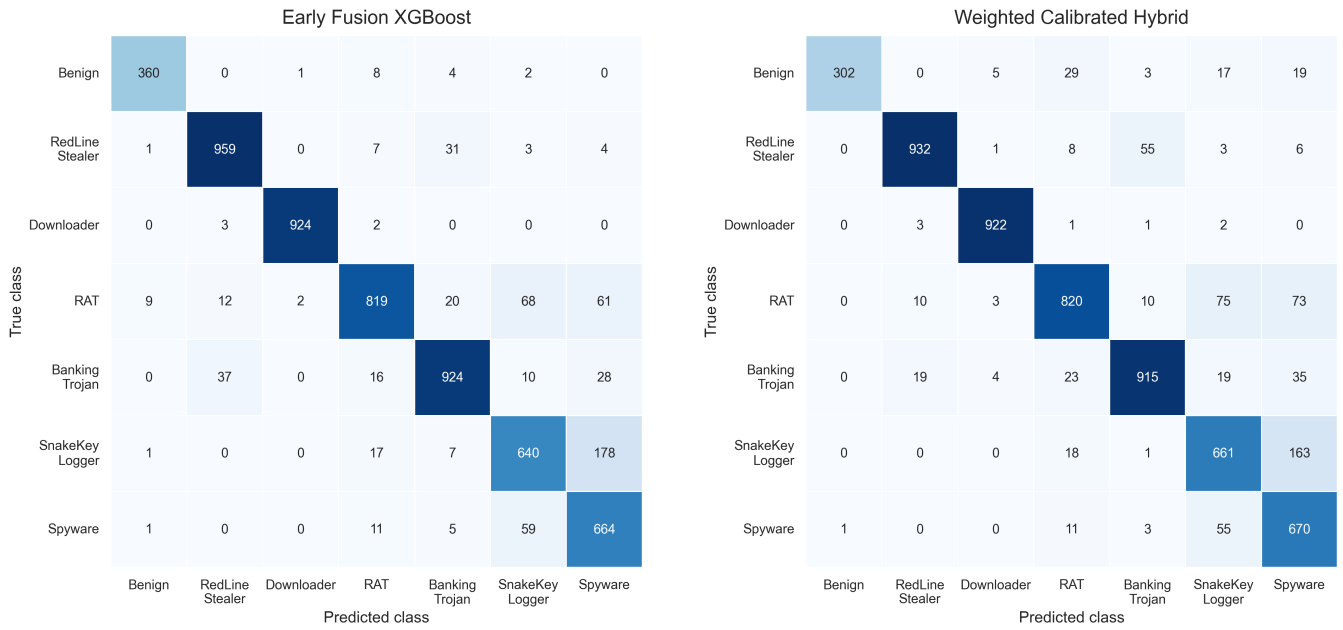


Figure 4. Multiclass confusion matrices for the two most informative final-system configurations on the corrected benchmark: early-fusion XGBoost (**left**) and the weighted calibrated hybrid (**right**).

Table 8. Execution time of the main phases in the corrected benchmark.

Phase	Time (s)
Preprocessing and feature reduction	816.98
Static-branch training	10.02
Dynamic-branch training	147.68
Weighted decision fusion	3.54
Early-fusion baseline training	19.69
Equal-weight late-fusion baseline	4.64

This result is useful for two reasons. First, it shows that the reduced dynamic model remains auditable after aggressive compression. Second, it reveals that many of the dominant signals correspond to low-level runtime, library, or environment-management calls rather than to immediately semantic malware capabilities. This is why the XAI output is treated as an audit artifact: it helps identify what the model depends on, but it does not by itself justify causal claims about attacker intent or malware functionality.

To complement the global audit views, Figure 7 reports a local LIME explanation for sample 1051, which corresponds to the only false negative of the weighted calibrated hybrid on the corrected benchmark. In this case, a true `Spyware` sample is classified as `Benign`, with decision-level probabilities 0.7868 for `Benign` and 0.2132 for `Malware`. The explanation shows that the local decision is driven by the joint absence or low activation of a small set of static and dynamic indicators, rather than by a diffuse contribution of many weak signals.

This local example is not presented as a systematic evaluation of explanation quality. Its value is diagnostic: it shows how the XAI audit layer can be used to inspect safety-critical errors and to understand which reduced signals contribute to a rare but operationally relevant failure mode.

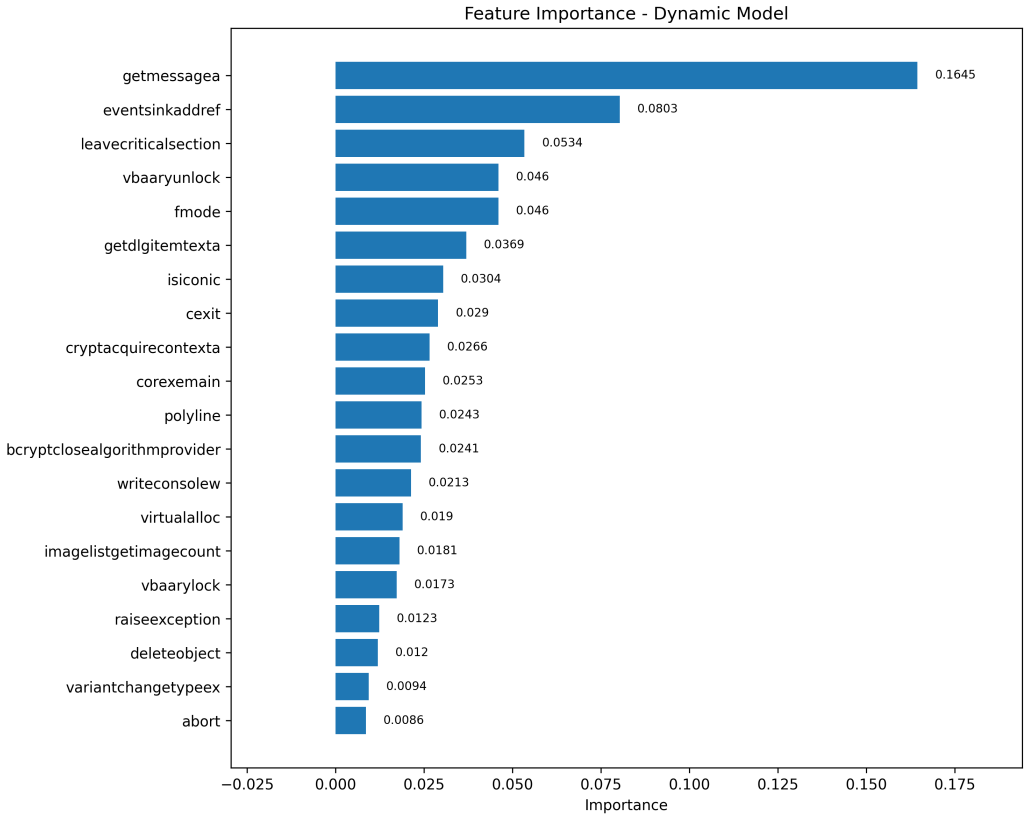


Figure 5. Global feature-importance ranking used as an XAI audit view of the calibrated dynamic branch on the corrected benchmark after sparse-aggressive reduction.

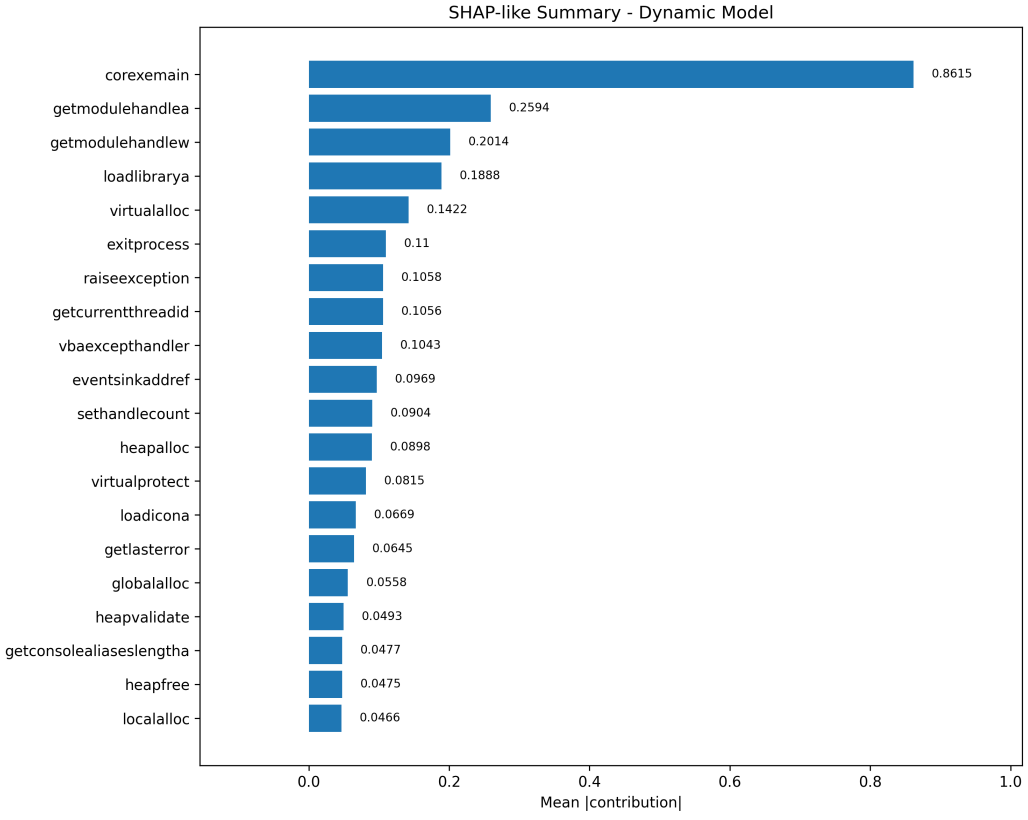


Figure 6. SHAP summary plot of the calibrated dynamic branch on the corrected benchmark after sparse-aggressive reduction.

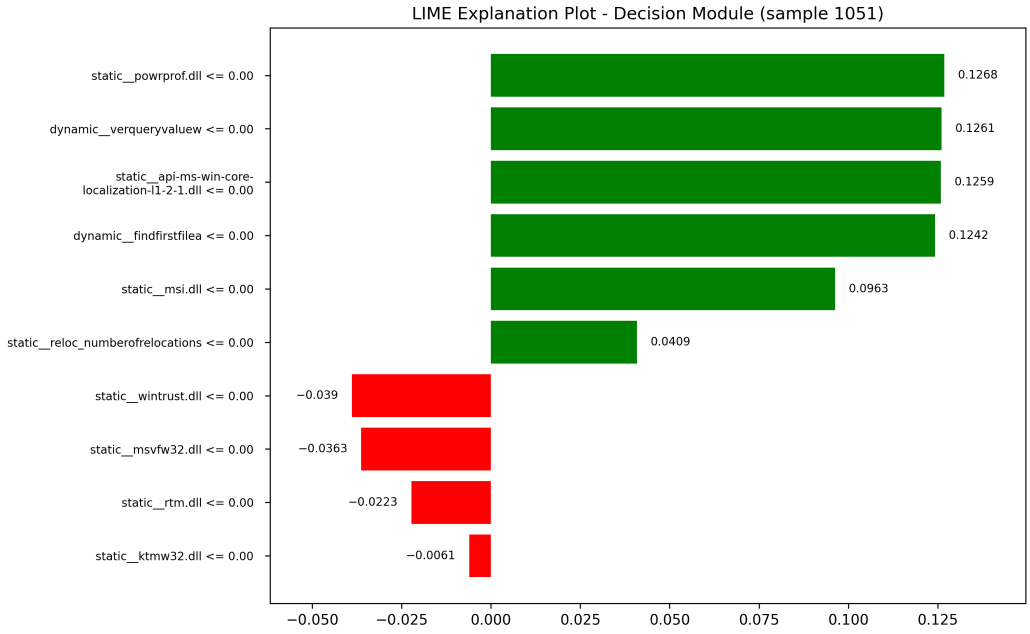


Figure 7. Local LIME explanation for sample 1051, the only false negative of the weighted calibrated hybrid on the corrected benchmark. The sample belongs to the `Spyware` class but is predicted as `Benign`.

6. Discussion

The discussion of the results leads to three main observations. First, after removing label leakage, the static branch remains strong but no longer near-perfect. This is a more credible outcome and reinforces the need for explicit leakage controls in malware-classification studies. Second, the dynamic branch is most useful as a complementary source of information rather than as the strongest standalone classifier. Its calibrated version improves the branch-level trade-off without increasing dimensionality, but the class-wise results also show that its behavior is uneven, especially on `SnakeKeyLogger` and `Spyware`. Third, the comparison with early fusion and equal-weight fusion shows that the value of the final system depends on the evaluation objective: early fusion is preferable when the goal is maximizing aggregate multiclass scores, whereas the weighted calibrated hybrid is preferable only under an operational criterion that explicitly prioritizes minimizing missed malware.

This trade-off is not merely numerical; it also clarifies the role of fusion in operational settings. Early fusion benefits from a single learner that can jointly exploit the reduced static and dynamic spaces, which helps explain its stronger aggregate multiclass performance. The weighted calibrated hybrid, however, preserves the identity of the two branches and gives more influence to the stronger branch without suppressing the contribution of the weaker one. In the corrected benchmark, this design is especially valuable because the dynamic branch contributes very high malware recall while the static branch stabilizes family-level discrimination and benign recognition. At the same time, the weighted hybrid does not dominate early fusion on every security metric: its `F2` remains slightly lower than early fusion, and its benign recall is clearly lower. The resulting system is, therefore, not a universal winner, but it is the configuration that best matches a false-negative-first security objective in which missed malware is treated as the dominant operational cost.

From an operational viewpoint, the computational profile is also informative. Preprocessing dominates total execution time, and the dynamic branch remains the most expensive trainable component even after aggressive reduction. By contrast, weighted decision fusion adds limited overhead, which supports its use as a practical decision-

support layer. From a deployment perspective, this cost is acceptable mainly because the preprocessing pipeline is offline and at dataset-level rather than part of per-alert online scoring. In a production setting, the reduction artifacts can be cached after feature extraction, sparse operations can be parallelized, and unchanged branch artifacts can be reused across retraining cycles. The practical bottleneck is, therefore, periodic batch preparation rather than real-time fusion, which remains lightweight.

The XAI audit layer should be interpreted at the same pragmatic level. As shown by the global ranking in Figure 5, by the SHAP summary in Figure 6, and by the local false-negative case in Figure 7, in this study the XAI layer contributes in three bounded ways: *(i)* it confirms that the reduced dynamic model relies on a concentrated subset of signals, *(ii)* it highlights when dominant predictors are generic runtime or library-level calls rather than directly interpretable malicious actions, and *(iii)* it enables post hoc inspection of selected branch and fused outputs, including rare operationally relevant errors. The contrast between impurity-based ranking and SHAP ordering is informative here: the exact feature hierarchy is not identical across methods, yet both views indicate that the model is not relying on diffuse noise across the retained space. The local LIME case adds a complementary perspective by showing how the same audit logic can be applied to a concrete failure mode. This makes the XAI layer useful for audit and debugging, but it should not be overstated: these explanations remain statistical rather than causal and are not validated through a dedicated analyst study.

Finally, the findings also provide a direct answer to the research questions posed in the introduction. RQ1 is addressed by the selection of an aligned benchmark in which static and dynamic evidence can support the same family-level task without artificial fusion constraints. RQ2 is addressed by the reduction and optimization experiments, which show that branch-specific preprocessing is essential, especially for the sparse dynamic block. RQ3 is addressed by the corrected benchmark comparison, which makes clear that the preferred final system depends on the evaluation objective, with the weighted calibrated hybrid emerging as the most suitable configuration only when the operational criterion explicitly prioritizes minimizing false negatives.

7. Limitations

Several limitations remain. The benchmark relies on a stratified random split rather than on a time-ordered split, and, therefore, it should not be interpreted as a realistic future-malware generalization test. A more deployment-oriented extension would order samples chronologically, train the two branches on the older portion of the corpus, tune calibration and fusion on a subsequent validation window, and reserve the most recent window for final testing, thereby exposing concept drift and family-evolution effects more directly. No explicit adversarial or evasion-oriented robustness evaluation is included. The work also uses pre-extracted public features instead of building a raw-binary or sandbox-instrumentation pipeline from scratch. In addition, although the XAI layer now includes both global and local inspection views, its usefulness is not validated through a dedicated analyst-centered study. These limitations help define the next steps of the research rather than weaken the internal validity of the controlled comparisons reported here.

8. Conclusions

This paper presented an explainable hybrid malware-classification pipeline built on an aligned public benchmark and evaluated through a progression of feature-reduction, dynamic-optimization, and fusion experiments. The study showed that benchmark choice is not a secondary implementation detail: the coherence of the static/dynamic dataset is a precondition for principled hybrid design. It also showed that sparse-aggressive

reduction makes the dynamic branch computationally manageable, that calibrated dynamic inference is the strongest dynamic variant among those tested, and that the choice of the final decision-fusion strategy depends on whether the priority is multiclass accuracy or operational malware recall. The XAI audit layer adds a complementary diagnostic view by showing that, after reduction, the dynamic branch depends on a concentrated subset of API-level signals, while also confirming that these explanations should be interpreted as model-inspection artifacts rather than as causal descriptions of malware behavior.

In this sense, the paper answers the three guiding research questions in a coherent sequence: first by identifying a benchmark that genuinely supports hybrid learning, then by showing the value of branch-specific reduction and dynamic optimization, and finally by demonstrating that the final-system choice should be driven by the intended operational objective rather than by a single aggregate metric. The take-home message is, therefore, straightforward: in explainable hybrid malware classification, methodological coherence matters more than algorithmic embellishment, and the preferred system should be chosen against the operational objective rather than a single aggregate score. If the priority is aggregate multiclass performance, the strongest baseline remains early fusion. If the priority is minimizing missed malware, the recommended configuration is the weighted calibrated hybrid system evaluated on the corrected no-leak benchmark, whose validation-derived branch weights yield malware recall of 0.9998 and only one false negative on the test set. Future work should extend the benchmark with temporal or out-of-distribution evaluation, adversarial robustness analysis, and formal validation of explanation usefulness for security analysts.

Author Contributions: Conceptualization, C.A., G.L., R.D.P., N.E., and A.M.; methodology, C.A., G.L., and R.D.P.; software, C.A., and G.L.; validation, C.A., G.L., R.D.P., N.E., and A.M.; writing—original draft preparation, G.L. and C.A.; writing—review and editing, C.A., G.L., R.D.P., N.E., and A.M.; supervision, C.A.; and project administration, C.A. and R.D.P. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the framework of the project "xTech NextHub: Competence Center for Innovative Solutions Development" through the Bando Regionale della Puglia per gli Aiuti in Esenzione 17 del 30/09/2014-BURP 139 suppl. del 06/10/2014 e s.m.i.-Titolo II Capo 2 del Regolamento Generale "Avviso per la Presentazione dei Progetti Promossi da Grandi Imprese ai Sensi Dell'articolo 17 del Regolamento".

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data used in this study are available from publicly accessible sources cited in the manuscript, in particular the Windows Malwares dataset version 1.0 [30]. The processed artifacts and experiment results supporting the conclusions of this work are available from the corresponding author on reasonable request.

Conflicts of Interest: Riccardo Di Pietro, at the time of carrying out this research, was affiliated with Deloitte Consulting. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest. The authors declare that this study was carried out within the framework of the xTech NextHub project, an advanced Competence Center in Bari, Italy, spearheaded by Deloitte, with regional backing from the Apulia Region and collaboration with local institutions. The project provided the broader research and innovation framework for the work. Deloitte Consulting was not involved in the study design, collection, analysis, interpretation of data, the writing of this article, or the decision to submit it for publication.This statement is also reflected in the authors' affiliation and funding information reported in the manuscript.

Appendix A. Full List of Literature Review Articles

Table A1. List of articles identified through the literature review. The second column marks with "Y" (for Yes) the articles that were selected and further analyzed.

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
ACM-001		Journal	2023	Malware Detection with Hybrid Datasets in Machine Learning Model Experiments	Thapa, Bipun; Lee, Tarie	J. Comput. Sci. Coll.	349
ACM-002		Conference	2024	Comprehensive Analysis of Consistency and Robustness of Machine Learning Models in Malware Detection	Kasarapu, Sreenitha; Bhusal, Dipkamal; Rastogi, Nidhi; Pudukotai Dinakarrao, Sai Manoj	Great Lakes Symposium on VLSI 2024	477–482
ACM-003	Y	Conference	2024	Building Visual Malware Dataset using VirusShare Data and Comparing Machine Learning Baseline Model to CoAtNet for Malware Classification	Bruzzese, Roberto	2024 16th International Conference on Machine Learning and Computing	185–193
ACM-004	Y	Conference	2022	Quo Vadis: Hybrid Machine Learning Meta-Model Based on Contextual and Behavioral Malware Representations	Trizna, Dmitrijs	15th ACM Workshop on Artificial Intelligence and Security	127–136
ACM-005		Conference	2022	An Early Detection of Android Malware Using System Calls based Machine Learning Model	Zhang, Xinrun; Mathur, Akshay; Zhao, Lei; Rahmat, Safia; Niyaz, Quamar; Javaid, Ahmad; Yang, Xiaoli	17th International Conference on Availability, Reliability and Security	
ACM-006	Y	Conference	2023	Decoding the Secrets of Machine Learning in Malware Classification: A Deep Dive into Datasets, Feature Extraction, and Model Performance	Dambra, Savino; Han, Yufei; Aonzo, Simone; Kotzias, Platon; Vitale, Antonino; Caballero, Juan; Balzarotti, Davide; Bilge, Leyla	2023 ACM SIGSAC Conference on Computer and Communications Security	60–74
ACM-007		Conference	2021	An Empirical Evaluation of Automated Machine Learning Techniques for Malware Detection	Kundu, Partha Pratim; Anatharaman, Lux; Truong-Huu, Tram	2021 ACM Workshop on Security and Privacy Analytics	75–81
ACM-008	Y	Conference	2019	Machine learning application for malwares classification using visualization technique	Ikram, Ben Abdel Ouahab; Mohammed, Bouhorma; Abdelhakim, Boudhir Anouar; Lotfi, El Aachak; Zafar, Bassam	4th International Conference on Smart City Applications	
ACM-009		Conference	2019	Vulnerability assessment of machine learning based malware classification models	Raju, Godwin; Zavarsky, Pavol; Makanju, Adetokunbo; Malik, Yasir	Genetic and Evolutionary Computation Conference Companion	1615–1618

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
ACM-010		Conference	2018	Evaluating Machine Learning Models for Android Malware Detection: A Comparison Study	Rana, Md. Shohel; Gudla, Charan; Sung, Andrew H.	2018 VII International Conference on Network, Communication and Computing	17–21
ACM-011		Journal	2020	Machine Learning Framework to Analyze IoT Malware Using ELF and Opcode Features	Tien, Chin-Wei; Chen, Shang-Wen; Ban, Tao; Kuo, Sy-Yen	Digital Threats	
ACM-012		Conference	2020	A Malware Detection Framework Based on Forensic and Unsupervised Machine Learning Methodologies	Irfan, Ahmad Naim; Ariffin, Aswami; Mahrin, Mohd Naz'ri; Anuar, Syahid	2020 9th International Conference on Software and Computer Applications	194–200
ACM-013		Conference	2018	Enhancing Machine Learning Based Malware Detection Model by Reinforcement Learning	Wu, Cangshuai; Shi, Jiangyong; Yang, Yuexiang; Li, Wenhua	8th International Conference on Communication and Network Security	74–78
ACM-014		Conference	2017	Dynamic Permissions based Android Malware Detection using Machine Learning Techniques	Mahindru, Arvind; Singh, Paramvir	10th Innovations in Software Engineering Conference	202–210
ACM-015		Conference	2018	A Novel Malware Analysis Framework for Malware Detection and Classification using Machine Learning Approach	Sethi, Kamalakanta; Chaudhary, Shankar Kumar; Tripathy, Bata Krishan; Bera, Padmalochan	19th International Conference on Distributed Computing and Networking	
ACM-016		Conference	2020	WOLF: automated machine learning workflow management framework for malware detection and other applications	Kiani, Sohaib; Awan, Sana; Huan, Jun; Li, Fengjun; Luo, Bo	7th Symposium on Hot Topics in the Science of Security	
ACM-017		Conference	2019	ATMPA: attacking machine learning-based malware visualization detection methods via adversarial examples	Liu, Xinbo; Zhang, Jiliang; Lin, Yaping; Li, He	International Symposium on Quality of Service	
ACM-018		Conference	2014	DroidDolphin: a dynamic Android malware detection framework using big data and machine learning	Wu, Wen-Chieh; Hung, Shih-Hao	2014 Conference on Research in Adaptive and Convergent Systems	247–252
IEEE-001		Conference	2010	Analysis of Machine learning Techniques Used in Behavior-Based Malware Detection	Firdausi, Ivan; lim, Charles; Erwin, Alva; Nugroho, Anto Satriyo	2010 Second International Conference on Advances in Computing, Control, and Telecommunication Technologies	201–203

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-002		Conference	2020	A Static Feature Selection-based Android Malware Detection Using Machine Learning Techniques	Sangal, Aviral; Verma, Harsh Kumar	2020 International Conference on Smart Electronics and Communication (ICOSEC)	48–51
IEEE-003		Conference	2017	Protecting data from malware threats using machine learning technique	Chowdhury, Mozammel; Rahman, Azizur; Islam, Rafiqul	2017 12th IEEE Conference on Industrial Electronics and Applications (ICIEA)	1691–1694
IEEE-004		Conference	2018	Detecting Java Compiled Malware using Machine Learning Techniques	Balan, Gheorghe; Popescu, Adrian Stefan	2018 20th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)	435–439
IEEE-005		Conference	2022	DynaMalDroid: Dynamic Analysis-Based Detection Framework for Android Malware Using Machine Learning Techniques	Haidros Rahima Manzil, Hashida; Naik S, Manohar	2022 International Conference on Knowledge Engineering and Communication Systems (ICKES)	1–6
IEEE-006		Conference	2016	Can malware analysts be assisted in their work using techniques from machine learning?	Novkovic, Igor; Gros, Stjepan	2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)	1408–1413
IEEE-007		Conference	2022	Machine Learning Techniques for Malware Detection- a Research Review	Jain, Prati; Rajvaidya, Ishita; Sah, Keshav Kumar; Kannan, Jayanthi	2022 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)	1–6
IEEE-008		Conference	2023	Malware Prediction And Classification For Android Applications Using Machine Learning Techniques	Kalpana, T	2023 International Conference on Computer Communication and Informatics (ICCCI)	1–7
IEEE-009		Conference	2024	Advanced Network Malware Detection: Integrating PySpark and Machine Learning Techniques	Dorrah, Mostafa; ElMaghraby, Asmaa; ElSaadany, Abdallah; Atta, Mohamed; Ashraf, Ahmed; Adel, Yousef; Yasser, Tawfik; Fathi, Mostafa; Abdelbaky, Ibrahim	2024 6th International Conference on Computing and Informatics (ICCI)	131–136
IEEE-010		Conference	2021	Malware Analysis using Ensemble Techniques: A Machine Learning Approach	Sharma, Sachin; Bharti, Santosh	2021 International Conference on Artificial Intelligence and Machine Vision (AIMV)	1–5
IEEE-011		Conference	2020	Analysis of Malware Prediction Based on Infection Rate Using Machine Learning Techniques	bin Asad, Ashub; Mansur, Raiyan; Zawad, Safir; Evan, Nahian; Hossain, Muhammad Iqbal	2020 IEEE Region 10 Symposium (TENSYMP)	706–709

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-012		Conference	2020	A Machine Learning Technique to Detect Behavior Based Malware	Chaudhary, Shubham; Garg, Anchal	2020 10th International Conference on Cloud Computing, Data Science & Engineering (Confluence)	655–659
IEEE-013		Conference	2023	Features Reduction with PCA Technique for Malware Detection Using Machine Learning	Mantoro, Teddy; Ginanjar, Agung; Ayu, Media Anugerah; Aditiawarman, Umar	2023 IEEE 9th International Conference on Computing, Engineering and Design (ICCED)	1–6
IEEE-014		Conference	2022	Detecting Android Malware and Sensitive Data Flows Using Machine Learning Techniques	Raman, Ramakrishnan; Nirmal, Khyati R.; Gehlot, Anita; Trivedi, Snehal; Sain, Dimple; Ponnusamy, R.	2022 5th International Conference on Contemporary Computing and Informatics (IC3I)	1694–1698
IEEE-015		Conference	2022	Android Malware Detection Using Machine Learning Techniques	Raghuvanshi, Priya; Singh, Jyoti Prakash	2022 International Conference on Computational Science and Computational Intelligence (CSCI)	1117–1121
IEEE-016		Conference	2022	ExFAURL Scanner: A Framework for Malware Detection in Binary files and URLs using Machine Learning Techniques	Ajwani, Juhi Kailash; Jain, Ranjanbala; Badhe, Abhishek; Prabhu, Jyeshtha; Narkhede, Atharva	2022 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)	1–5
IEEE-017	Y	Conference	2022	Development of Malware Classification Technique Based on Machine Learning Neural Network and High Order Markov Chain	Gaol, Devani Claudia Lumban; Hendrawan; Ismail, Setia Juli Irzal	2022 8th International Conference on Wireless and Telematics (ICWT)	1–6
IEEE-018		Conference	2023	Effects of Malware Detection Parameters on Classical vs. Deep ML Techniques	Vhito, Tadiwa; Suaboot, Jakapan; Werapun, Warodom; Kliangsuwan, Thitinan	2023 7th International Conference on Information Technology (InCIT)	440–445
IEEE-019		Conference	2015	Machine Learning Based Hybrid Behavior Models for Android Malware Analysis	Chuang, Hsin-Yu; Wang, Sheng-De	2015 IEEE International Conference on Software Quality, Reliability and Security	201–206
IEEE-020		Conference	2020	Evaluating Performance Maintenance and Deterioration Over Time of Machine Learning-based Malware Detection Models on the EMBER PE Dataset	Galen, Colin; Steele, Robert	2020 Seventh International Conference on Social Networks Analysis, Management and Security (SNAMS)	1–7
IEEE-021		Journal	2023	Are Machine Learning Models for Malware Detection Ready for Prime Time?	Cavallaro, Lorenzo; Kinder, Johannes; Pendlebury, Feargus; Pierazzi, Fabio	IEEE Security & Privacy	53–56

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-022		Conference	2019	Predicting and Preventing Malware in Machine Learning Model	Nisha, D; Sivaraman, E; Honnavalli, Prasad B	2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)	1–7
IEEE-023		Conference	2021	Identify Type of Android Malware with Machine Learning Based Ensemble Model	Arslan, Recep Sinan	2021 5th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)	628–632
IEEE-024		Conference	2022	A comparative study of machine learning models for malware detection	Mushtaq, Earum; Shahid, Farah; Zameer, Aneela	2022 19th International Bhurban Conference on Applied Sciences and Technology (IBCAST)	677–681
IEEE-025		Conference	2020	Experimental Comparison of Machine Learning Models in Malware Packing Detection	Kim, Jong-Wouk; Namgung, Juhong; Moon, Yang-Sae; Choi, Mi-Jung	2020 21st Asia-Pacific Network Operations and Management Symposium (APNOMS)	377–380
IEEE-026		Conference	2023	Hybrid Multimodal Machine Learning Driven Android Malware Recognition and Classification Model	A, Arthi.; Aggarwal, Kapil; Karthikeyan, R; Kayalvili, S.; S, Srimathi; Srivastava, Aviral	2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA)	1555–1560
IEEE-027		Conference	2023	An Improvised Machine Learning Model KNN for Malware Detection and Classification	Rao, M. Raj Shekhar; Yadav, Deepanshu; Anbarasu, V.	2023 International Conference on Computer Communication and Informatics (ICCCI)	1–4
IEEE-028		Conference	2022	Performance Enhancement of SVM-based ML Malware Detection Model Using Data Preprocessing	Singh, Priyanka; Borgohain, Samir Kumar; Kumar, Jayendra	2022 2nd International Conference on Emerging Frontiers in Electrical and Electronic Technologies (ICEFEET)	1–4
IEEE-029	Y	Conference	2021	A Comprehensive Survey on Identification of Malware Types and Malware Classification Using Machine Learning Techniques	Pachhala, Nagababu; Jothilakshmi, S.; Battula, Bhanu Prakash	2021 2nd International Conference on Smart Electronics and Communication (ICOSEC)	1207–1214
IEEE-030		Conference	2023	Leveraging Oversampling Techniques in Machine Learning Models for Multiclass Malware Detection in Smart Home Applications	Chowdhury, Abdullahi; Islam, Mohammad Manzurul; Kaisar, Shahriar; Khoda, Mahbub E; Naha, Ranesh; Khoshkholghi, Mohammad Ali; Aiash, Mahdi	2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)	2216–2221

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-031		Journal	2023	Detection of Android Malware Using Machine Learning and Siamese Shot Learning Technique for Security	Almarshad, Fahdah A.; Zakariah, Mohammed; Gashgari, Ghada Abdalaziz; Aldakheel, Eman Abdullah; Alzahrani, Abdullah I. A.	IEEE Access	127,697–127,714
IEEE-032		Conference	2020	Malware Analysis using Machine Learning and Deep Learning techniques	Patil, Rajvardhan; Deng, Wei	2020 SoutheastCon	1–7
IEEE-033		Conference	2023	A Comparison Study to Detect Malware using Deep Learning and Machine learning Techniques	Bokolo, Biodoumoye; Jinad, Razaq; Liu, Qingzhong	2023 IEEE 6th International Conference on Big Data and Artificial Intelligence (BDAI)	1–6
IEEE-034		Conference	2021	Malware Detection and Prevention using Artificial Intelligence Techniques	Hossain Faruk, Md Jobair; Shahriar, Hossain; Valero, Maria; Barsha, Farhat Lamia; Sobhan, Shahriar; Khan, Md Abdullah; Whitman, Michael; Cuzzocrea, Alfredo; Lo, Dan; Rahman, Akond; Wu, Fan	2021 IEEE International Conference on Big Data (Big Data)	5369–5377
IEEE-035		Journal	2023	A Lightweight and Multi-Stage Approach for Android Malware Detection Using Non-Invasive Machine Learning Techniques	Costa, Leonardo da; Moia, Vitor	IEEE Access	73,127–73,144
IEEE-036		Conference	2019	EDIMA: Early Detection of IoT Malware Network Activity Using Machine Learning Techniques	Kumar, Ayush; Lim, Teng Joon	2019 IEEE 5th World Forum on Internet of Things (WF-IoT)	289–294
IEEE-037		Conference	2019	A Survey on Android Malware Detection Techniques Using Machine Learning Algorithms	Alqahtani, Ebtesam J.; Zagrouba, Rachid; Almuhaideb, Abdullah	2019 Sixth International Conference on Software Defined Systems (SDS)	110–117
IEEE-038		Conference	2019	TFDroid: Android Malware Detection by Topics and Sensitive Data Flows Using Machine Learning Techniques	Lou, Songhao; Cheng, Shaoyin; Huang, Jingjing; Jiang, Fan	2019 IEEE 2nd International Conference on Information and Computer Technologies (ICICT)	30–36
IEEE-039		Conference	2020	A Comparative Analysis of Machine Learning Techniques for Classification and Detection of Malware	Al-Janabi, Maryam; Altamimi, Ahmad Mousa	2020 21st International Arab Conference on Information Technology (ACIT)	1–9
IEEE-040		Conference	2018	Accuracy Improved Malware Detection Method using Snort Sub-signatures and Machine Learning Techniques	Khammas, Ban M.; Hasan, Sami; Ahmed, Rabah Abood; Bassi, Joseph Stephen; Ismail, Ismahani	2018 10th Computer Science and Electronic Engineering (CEEC)	107–112

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-041		Conference	2018	New Techniques in Profiling Big Datasets for Machine Learning with a Concise Review of Android Mobile Malware Datasets	Canbek, Gurol; Sagiroglu, Seref; Taskaya Temizel, Tugba	2018 International Congress on Big Data, Deep Learning and Fighting Cyber Terrorism (IBIGDELFT)	117–121
IEEE-042		Conference	2022	A Literature Review on Android Mobile Malware Detection using Machine Learning Techniques	Sk, Heena Kauser.; V, Maria Anu.	2022 6th International Conference on Computing Methodologies and Communication (ICCMC)	986–991
IEEE-043		Conference	2016	Reverse engineering smart card malware using side channel analysis with machine learning techniques	Tsague, Hippolyte Djonon; Twala, Bheki	2016 IEEE International Conference on Big Data (Big Data)	3713–3721
IEEE-044	Y	Conference	2019	Static PE Malware Type Classification Using Machine Learning Techniques	Zhang, Shao-Huai; Kuo, Cheng-Chung; Yang, Chu-Sing	2019 International Conference on Intelligent Computing and its Emerging Applications (ICEA)	81–86
IEEE-045		Conference	2021	A Survey On Detecting Android Malware Using Machine Learning Technique	Priyadharshini, S; Shanthi, S	2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)	1621–1627
IEEE-046		Conference	2019	Performance comparison of local versus cloud malware detection on Android using machine learning techniques	Guyton, Fred	2019 SoutheastCon	1–8
IEEE-047		Conference	2022	Detection of Malware in the Network Using Machine Learning Techniques	Yogesh, B.; Reddy, G.Suresh	2022 International Conference on Recent Trends in Microelectronics, Automation, Computing and Communications Systems (ICMACC)	204–211
IEEE-048		Conference	2022	Malware Analysis Using Machine Learning Techniques	Kinger, Shakti; Reddy, Bhavanam Vijayabhaskar; Jadhao, Sanket; Hambarde, Kaustubh; Hullur, Aamir	2022 2nd International Conference on Intelligent Technologies (CONIT)	1–9
IEEE-049	Y	Conference	2021	Malware Classification Approaches Using Machine Learning Techniques: A Review	Naik, Shivarti; Dessai, Amita	2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT)	111–117

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-050		Conference	2021	Malware Detection Approaches using Machine Learning Techniques- Strategic Survey	Verma, Ashwini Kumar; Sharma, Sanjay Kumar	2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)	1958–1962
IEEE-051		Conference	2022	A Node-Embedding Features Based Machine Learning Technique for Dynamic Malware Detection	Rai, Sudhir Kumar; Mittal, Ashish; Mittal, Sparsh	2022 IEEE Conference on Dependable and Secure Computing (DSC)	1–8
IEEE-052		Conference	2023	Malware Detection in Android Based Devices by a Hybrid Approach Using Machine Learning Techniques	H Sai Sriraj, M V; Thambi, Krotha Kayne; Venkat Teja, Bacham Sai; Woonna, Venkat Amith	2023 3rd Asian Conference on Innovation in Technology (ASIANCON)	1–9
IEEE-053		Conference	2023	Design of Machine Learning-Based Malware Detection Techniques in Smartphone Environment	Baviskar, Pallavi V.; Singh, Guddi; Patil, Vijay Narendranath	2023 International Conference for Advancement in Technology (ICONAT)	1–5
IEEE-054		Conference	2023	A Systematic Review and Future Perspective of Android Malware Detection Based Machine Learning Techniques	Kaithal, Praveen Kumar; Sharma, Varsha	2023 IEEE International Conference on ICT in Business Industry & Government (ICTBIG)	1–17
IEEE-055		Conference	2023	A Survey of Traditional and Machine Learning-based Malware Detection Techniques	Zhang, Yuting; Gu, Yu	2023 IEEE 3rd International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA)	1373–1378
IEEE-056	Y	Conference	2024	Machine Learning Enables Malware Detection and Classification Techniques	Rana, Deepak Singh; Dimri, Sushil Chandra	2024 IEEE International Conference on Computing, Power and Communication Technologies (IC2PCT)	1215–1221
IEEE-057		Conference	2022	Predictive Analysis of Malware using Machine Learning Techniques	Kinger, Shakti; Hambarde, Kaustubh ManojKumar	2022 International Conference on Smart Generation Computing, Communication and Networking (SMART GENCON)	1–8
IEEE-058		Conference	2023	Machine Learning Techniques for Permission-based Malware Detection in Android Applications	Khan, Inam Ullah; Ali Khan, Zahoor; Ahmad, Mir; Khan, Ali Haider; Muahmmad, Fida; Imran, Azhar; Ahmed, Sheeraz; Hamid, Muhammad Khalid	2023 9th International Conference on Information Technology Trends (ITT)	7–13

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-059		Conference	2023	Detection of Malware Using Machine Learning techniques on Random Forest, Decision Tree, KNeighbour, AdaBoost, SGD, ExtraTrees and GaussianNB Classifier	Gill, Kanwarpartap Singh; Anand, Vatsala; Gupta, Rupesh; Hsiung, Pao-Ann	2023 4th IEEE Global Conference for Advancement in Technology (GCAT)	1–7
IEEE-060		Journal	2023	Gauss-Mapping Black Widow Optimization With Deep Extreme Learning Machine for Android Malware Classification Model	Aldehim, Ghadah; Arasi, Munya A.; Khalid, Majdi; Aljameel, Sumayh S.; Marzouk, Radwa; Mohsen, Heba; Yaseen, Ishfaq; Ibrahim, Sara Saadeldeen	IEEE Access	87,062–87,070
IEEE-061		Journal	2024	PDF Malware Detection: Toward Machine Learning Modeling With Explainability Analysis	Hossain, G. M. Sakhawat; Deb, Kaushik; Janicke, Helge; Sarker, Iqbal H.	IEEE Access	13,833–13,859
IEEE-062		Conference	2015	Malware detection via API calls, topic models and machine learning	Sundarkumar, G. Ganesh; Ravi, Vadlamani; Nwogu, Ifeoma; Govindaraju, Venu	2015 IEEE International Conference on Automation Science and Engineering (CASE)	1212–1217
IEEE-063		Conference	2020	Malware Detection in Android Systems with Traditional Machine Learning Models: A Survey	Bayazit, Esra Calik; Koray Sahingoz, Ozgur; Dogan, Buket	2020 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)	1–8
IEEE-064		Conference	2022	A Comparison of Machine and Deep Learning Models for Detection and Classification of Android Malware Traffic	Bovenzi, Giampaolo; Cerasuolo, Francesco; Montieri, Antonio; Nascita, Alfredo; Persico, Valerio; Pescape, Antonio	2022 IEEE Symposium on Computers and Communications (ISCC)	1–6
IEEE-065		Conference	2014	CrowdSource: Automated inference of high level malware functionality from low-level symbols using a crowd trained machine learning model	Saxe, Joshua; Turner, Rafael; Blokhin, Kristina	2014 9th International Conference on Malicious and Unwanted Software: The Americas (MALWARE)	68–75
IEEE-066		Conference	2022	Android malware detection: Investigating the impact of imbalanced datasets on the performance of machine learning models	Sawadogo, Zakaria; Mendy, Gervais; Dembele, Jean Marie; Ouya, Samuel	2022 24th International Conference on Advanced Communication Technology (ICACT)	435–441
IEEE-067	Y	Conference	2021	Malware Detection and Classification Model Using Machine Learning Random Forest Approach	Irawan, Carti; Mantoro, Teddy; Ayu, Media Anugerah	2021 IEEE 7th International Conference on Computing, Engineering and Design (ICCED)	1–5

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-068		Conference	2017	Machine Learning based Malware Detection - How to Balance Memory Footprint with Model Accuracy	Gavrilut, Dragos Teodor; Anton, Dan Gabriel; Popoiu, George	2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)	232–238
IEEE-069		Conference	2023	Android malware detection: An in-depth investigation of the impact of the use of imbalance datasets on the efficiency of machine learning models	Sawadogo, Zakaria; Dembele, Jean-Marie; Mendy, Gervais; Ouya, Samuel	2023 25th International Conference on Advanced Communication Technology (ICACT)	1460–1467
IEEE-070		Conference	2023	Obfuscated Malware detection using Machine Learning models	Balasubramanian, Karthik Manikandan; Vasudevan, Shri Venkatakrishnan; Thangavel, Senthil Kumar; T, Gireesh Kumar; Srinivasan, Kartik; Tibrewal, Anjali; Vajipayajula, Sulakshan	2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)	1–8
IEEE-071		Conference	2022	Hybrid Android Malware Detection Model using Machine learning Algorithms	Lee Yam, Andrew Kobe T.; Ballesta, John Michael R.; Lanceta, Joyce Ann H.; Mogol, Moises Kairon T.; Labanan, Rogel	2022 2nd International Conference in Information and Computing Research (iCORE)	66–71
IEEE-072		Conference	2023	Keylogger Malware Detection Using Machine Learning Model for Platform-Independent Devices	Srinivasan, Kartik	2023 International Conference on Emerging Trends in Networks and Computer Communications (ETNCC)	1–6
IEEE-073		Conference	2023	Interpreting Machine and Deep Learning Models for PDF Malware Detection using XAI and SHAP Framework	Rahman, Tahsinur; Ahmed, Nusaiba; Monjur, Shama; Haque, Fasbeer Mohammad; Hossain, Muhammad Iqbal	2023 2nd International Conference for Innovation in Technology (INOCON)	1–9
IEEE-074		Conference	2024	Identifying Malware using Machine Learning Ensemble Model	Bandlapalli, Saketh; Janarthan, S Nikhil; Ragul, S; Sujatha, G.	2024 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI)	1–6
IEEE-075		Conference	2022	Clean-label Backdoor Attack on Machine Learning-based Malware Detection Models and Countermeasures	Zheng, Wanjia; Omote, Kazumasa	2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)	1235–1242

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-076		Conference	2022	Evaluating the effort of building a Machine Learning model for malware detection from ground zero	Anton, Dan-Gabriel	2022 24th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)	365–370
IEEE-077		Conference	2023	Malware Detection Using XGBoost based Machine Learning Models - Review	Hariharan, B.; Siva, R.; Sadagopan, S.; Mishra, Vaibhav; Raghav, Yash	2023 2nd International Conference on Edge Computing and Applications (ICECAA)	964–970
IEEE-078		Conference	2024	Optimized Model Identification for Malware Detection in a File Using Machine Learning	N, Laya; Shetty J, Shruthi; H, Sarojadevi; P, Sowmya	2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)	1–8
IEEE-079		Conference	2023	Malware detection using Explainable ML models based on Feature Extraction using API calls	Banda, Bhanu Prakash Reddy; Govan, Bianca; Roy, Kaushik; Bryant, Kelvin	2023 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD)	1–7
IEEE-080		Conference	2024	Towards Explainability Using ML And Deep Learning Models For Malware Threat Detection	Snehal, Mattaparti Satya Chandana; Nagoor, Veeraboina; Rohit, Sugrivu; Raghunandan, S; Thangavel, Senthil Kumar; Srinivasan, Kartik; Kapoor, Pratyul	2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)	1–6
IEEE-081		Conference	2022	Explainable AI for Android Malware Detection: Towards Understanding Why the Models Perform So Well?	Liu, Yue; Tantithamthavorn, Chakkrit; Li, Li; Liu, Yepang	2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)	169–180
IEEE-082		Conference	2023	Debugging Malware Classification Models Based on Event Logs with Explainable AI	Gwak, Joon-Young; Wakodikar, Priti; Wang, Meng; Yan, Guanhua; Shu, Xiaokui; Stoller, Scott D.; Yang, Ping	2023 IEEE International Conference on Data Mining Workshops (ICDMW)	939–948
IEEE-083	Y	Conference	2025	Hybrid AI-Based Static Analysis for Malware Detection: A Feature Engineering and Model Optimization Approach	Abdrakhmanov, Bakdaulet	2025 IEEE 5th International Conference on Smart Information Systems and Technologies (SIST)	1–6
IEEE-084		Conference	2025	Hybrid Attention-Driven AI Model for Malware Attack Detection in Consumer Electronic Devices	Gupta, Brij B.; Gaurav, Akshat; Castiglione, Arcangelo; Arya, Varsha; Chui, Kwok Tai	2025 IEEE International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan)	125–126

Table A1. *Cont.*

ID	Selected	Type	Year	Title	Authors	Proceedings/Journal	Pages
IEEE-085		Conference	2025	My Model is Malware to You: Transforming AI Models into Malware by Abusing TensorFlow APIs	Zhu, Ruofan; Chen, Ganhao; Shen, Wenbo; Xie, Xiaofei; Chang, Rui	2025 IEEE Symposium on Security and Privacy (SP)	486–503
IEEE-086		Conference	2025	Deciphering Model Decisions in Android Malware Detection with Explainable AI	Sayed, Moinul; Anbar, Amreen; Saha, Sajal; Haque, Anwar	2025 International Symposium on Networks, Computers and Communications (ISNCC)	1–6
IEEE-087	Y	Conference	2025	Next-Gen Malware Detection using AI: AI-Powered Malware Threat Detection and Automated Malware Classification Through Machine Learning	Manikandan, K P; Reddy Onteddu, Navaneeswar; Chilimi, Abdul Kalam	2025 International Conference on Intelligent Computing and Control Systems (ICICCS)	581–588
IEEE-088	Y	Conference	2025	Real-Time Detection of Polymorphic Malware Using AI and Synthetic Data Augmentation	Pallavi, Pallavi; Joshi, Sandeep; Bairwa, Amit Kumar	2025 XXX International Conference on Information, Communication and Automation Technologies (ICAT)	1–6
IEEE-089		Conference	2025	A Review Paper of Threats and an AI-Powered Malware Detection Model	Harshan, Mv; Baweja, Deeksha	2025 International Conference on Smart & Sustainable Technology (INCSST)	1–7
IEEE-090		Conference	2025	Optimization of Class Imbalance Techniques in Machine Learning Models for Network Intrusion Detection	Xie, Huijie; Shao, Yunlong; Li, Zhida; Alomari, Zakaria; Makanju, Adetokunbo	2025 9th International Conference on Cryptography, Security and Privacy (CSP)	102–106
IEEE-091		Conference	2025	EvadeSafe-XAI: A Robust Malware Defense System using Explainable AI	Ramya, Pendli; Divya, Kundaram; Kurmala, Bhavya Sri; Poranki, Vaishnavi Maitreyi; Kiran, Medikonda Asha; Nagamani, P.	2025 5th International Conference on Intelligent Technologies (CONIT)	1–7
IEEE-092	Y	Conference	2025	Malware Analysis Using Hashing and Explainable AI: A Comparative Study of LIME and SHAP Techniques	Pradhan, Ujan; M, Navaneeth K; Aditya, M N	2025 1st International Conference on AIML-Applications for Engineering & Technology (ICAET)	1–6

Appendix B. Analysis of Selected Articles

Table A2. Information extracted using the analysis framework applied to the 14 articles selected in the literature review. The complete reference of each article is available in Appendix A, Table A1, and in the reference list.

ID	Algorithm	Usage	Dataset	Feature Engineering	Evaluation Metrics	Distinctive Characteristics	Further Notes
ACM-003 [17]	CoAtNet, CNN	Malware image classification	VirusShare_00251 (65,536 samples)	Images generated from the binary representation of malware files (PNG)	Accuracy (CNN: 92.06%, CoAtNet: 94%), training loss, validation accuracy	CoAtNet combines convolutions and self-attention; binary-to-image transformation for visual analysis	Data augmentation negatively affects performance; CoAtNet outperforms CNN in terms of accuracy and generalization, especially on minority classes
ACM-004 [19]	Hybrid model (FFNN, CNN, GBDT)	Malware classification based on hybrid representations	Proprietary dataset: 98,966 samples (train/validation), 27,500 samples (test); 7 malware and benign families	Static properties (PE structure), dynamic properties (API call sequences), contextual properties (execution-time file path)	Accuracy, F1-score, precision, recall, AUC	Modular architecture with a meta-model combining static, dynamic, and contextual representations; use of Speakeasy emulators	Higher accuracy than individual modules; combining file path and API calls outperforms purely static approaches; robustness against adversarial attacks; pretrained models and emulated dataset publicly released
ACM-006 [5]	Random Forest, XGBoost	Binary and family-level classification	VirusTotal feed: 118,111 samples, 670 families	Static features (bytegrams, opcodegrams, headers, strings); dynamic features (API calls, network, files, processes, registry, threads, mutexes)	Accuracy, precision, recall, F1-score	Separation between static and dynamic features; analysis of packer/protector impact and missing values	Static features dominate overall performance, while dynamic features improve interpretability and robustness; combined models yield marginal improvements
ACM-008 [31]	K-Nearest Neighbors (KNN)	Malware image-family classification	MalImg dataset: 9339 samples across 25 families	Malware images generated from binary files; GIST descriptor used for feature extraction	Accuracy (97.92%)	Use of the GIST descriptor for malware-image representation; optimal value $K = 10$	Computer-vision techniques applied to malware analysis represent an innovative approach compared to prior literature

Table A2. *Cont.*

ID	Algorithm	Usage	Dataset	Feature Engineering	Evaluation Metrics	Distinctive Characteristics	Further Notes
IEEE-017 [18]	Neural networks, High-Order Markov Chains	Image-based malware detection and classification	MalImg dataset (9339 samples)	Transformation of malware binaries into image representations	Accuracy, precision, recall, F1-score	Use of high-order Markov chains to generate visual representations	Visual analysis and Markov-based techniques are robust against obfuscation techniques such as packers
IEEE-029 [3]	Systematic review of multiple algorithms	Review of ML techniques for malware detection and classification	Various datasets (VirusTotal, VXHeaven, etc.)	Discussion of static, dynamic, and hybrid features: API calls, opcodes, instruction sequences	Precision, recall, accuracy (across analyzed studies)	Comparison between machine learning and deep learning models for malware analysis	Review highlighting current trends and limitations; emphasis on advanced techniques such as Gradient Boosting, Random Forest, and clustering models
IEEE-044 [15]	Linear SVC, Logistic Regression, LightGBM, Random Forest	Malware-type classification on PE files	Customized EMBER dataset	Statically extracted features: byte histogram, byte-entropy histogram, string information, headers, imported/exported functions	Micro-averaged F1-score: 0.96 (Random Forest), macro-averaged F1-score: 0.89	Dataset relabeled using VirusTotal for multiclass classification; Random Forest achieves the best trade-off between accuracy and efficiency	Classifier optimized via grid search; recall issues for underrepresented families; proposal of class-weight balancing strategies
IEEE-049 [4]	Systematic review	Review of malware detection and classification methods using ML	Various datasets (MalImg, BIG2015, VirusTotal, VXHeaven)	Static (byte-entropy, opcodes, strings, API calls), dynamic (registry, network, mutex, file activity), and visual (grayscale images) features	Precision, recall, accuracy (method-dependent)	Detailed comparison of static, dynamic, and hybrid features; includes image-based malware-visualization techniques	Highlights advantages and limitations of different approaches; provides a comprehensive analysis of feature-extraction methods and recent network architectures
IEEE-056 [20]	Random Forest, Logistic Regression, Neural Network, KNN	Malware detection and classification	Microsoft Malware Classification Challenge (2015)	Static features (byte sequences, API calls, opcode patterns); dynamic features (system calls, network interactions)	Accuracy, ROC curve, detection time	Comparison of supervised-learning techniques; neural networks and logistic regression show faster inference	Analysis of temporal performance; ROC curves used for fine-grained evaluation of classification quality

Table A2. *Cont.*

ID	Algorithm	Usage	Dataset	Feature Engineering	Evaluation Metrics	Distinctive Characteristics	Further Notes
IEEE-067 [16]	Random Forest	Malware detection and classification	Microsoft Malware Classification Challenge	Static features (API calls, opcodes, headers, strings)	Accuracy, precision, recall, F1-score	Random Forest optimized via parameter tuning; focus on static features for computational efficiency	Evaluation confirms the effectiveness of static features for fast classification; model demonstrates robustness and high accuracy compared to more complex approaches
IEEE-083 [7]	Supervised ML (Logistic Regression, Random Forest, MLP, XGBoost, LightGBM, Decision Tree) with feature selection (RFE) and interpretability (SHAP)	Static PE malware detection (benign vs. malicious) from PE structural indicators	11,000 PE files from VirusTotal (2017–2024); 80/20 train-test split; 5-fold cross-validation	54 static PE-derived features (DOS/PE header fields, sections, etc.); RFE reduced to 17 features; SHAP used to analyze feature impact	Accuracy, Precision, Recall, F1-score (also stability via cross-validation); best reported accuracy 99.09% (XGBoost)	Strong focus on feature engineering + model optimization across multiple classifiers; SHAP used for global interpretability of the chosen model	Scope limited to static PE features and the specific collected dataset (VirusTotal); intended as an efficient static pipeline
IEEE-087 [8]	Supervised ML + DL comparison (Decision Tree, Random Forest, KNN, SVM, GBDT, CNN, ISGD) with hyperparameter tuning	Real-time malware classification emphasizing computational efficiency	Balanced dataset of 100,000 records (50,000 malware/ 50,000 benign) with 35 system-level behavioral features; 70/30 stratified split	Feature selection via ANOVA F-test (SelectKBest) selecting top-12 features; correlation heatmap to remove highly correlated features ($r > 0.9$); outlier removal mentioned	Accuracy, Precision, Recall, F1-score; plus execution time and model complexity analysis; DT/RF/KNN reported near-100% accuracy; CNN 97.79% with higher cost	Emphasis on feature selection + model choice (instead of dataset reduction) to keep accuracy high while targeting low overhead for real-time use	Dataset described as "system-level behavioral" but feature semantics are not deeply characterized in the paper; results appear extremely high, worth cautious interpretation

Table A2. *Cont.*

ID	Algorithm	Usage	Dataset	Feature Engineering	Evaluation Metrics	Distinctive Characteristics	Further Notes
IEEE-088 [6]	Deep Learning hybrid: CNN for static features + RNN/LSTM for dynamic sequences; GAN-based synthetic data augmentation	Real-time detection of polymorphic malware in enterprise/IoT networked environments (traffic + host/malware behavior)	Traffic logs from public sources (e.g., CICIDS 2017) + simulated data; also malware samples from VirusTotal/MalwareBazaar plus GAN-generated synthetic variants; explicit figures: 10,000 traffic records and 1500 polymorphic malware samples	Hybrid feature set: network traffic features (packet size, protocol, inter-arrival time, source/destination IPs), static malware features (file headers, entropy, file size), dynamic features (system calls, API call sequences); PCA mentioned for feature selection on traffic	Accuracy, Precision, Recall, F1-score, False Positive Rate; operational metrics: detection latency and scalability (packets/sec); reported: 96.3% accuracy, 4.3% FPR, 0.45 s latency, 50,000 packets/sec	Distinctive use of GAN augmentation to address labeled-data scarcity and improve generalization to unseen polymorphic variants; explicit "real-time + scalable" positioning	Evaluation described in a controlled/simulated environment; the "hybrid" notion is multi-source (traffic + malware samples) and multi-model (CNN+LSTM)
IEEE-092 [9]	Supervised ML with explainability: Random Forest classification + XAI comparison (LIME vs. SHAP); hashing-based feature extraction	Malware detection/classification with post hoc explanations to compare LIME and SHAP behavior	Malware samples sourced from VirusShare and VirusTotal; benign executables also used (dataset size not explicitly quantified in the extracted sections)	Opcode-sequence-based processing with hashing; preprocessing includes normalization and binarization; explanations generated and compared using LIME and SHAP	Accuracy, Precision, Recall, F1-score; reported: Accuracy 98.03%, Precision 98.14%, Recall 98.11%, F1 98.12	Comparative focus on XAI methods (LIME vs. SHAP) on the same pipeline; aims to show how explanation outputs differ for malware decisions	Paper is oriented to the XAI comparison more than to dataset rigor; dataset characterization is limited in the described sections

References

1. Massaro, A.; Epicoco, N.; Loseto, G.; Ardito, C.A. AI-Based Detection and Simulation of Hardware Trojan Attacks in Industrial Applications. *IEEE Access* **2025**, *13*, 50967–50979.
2. AV-TEST Institute. Malware Statistics & Trends Report. 2025. Available online: https://www.av-test.org/en/statistics/malware/ (accessed on 5 February 2026).
3. Pachhala, N.; Jothilakshmi, S.; Battula, B.P. A Comprehensive Survey on Identification of Malware Types and Malware Classification Using Machine Learning Techniques. In Proceedings of the 2021 2nd International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India, 7–9 October 2021; pp. 1207–1214. https://doi.org/10.1109/ICOSEC51865.2021.9591763.
4. Naik, S.; Dessai, A. Malware Classification Approaches Using Machine Learning Techniques: A Review. In Proceedings of the 2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT), Mysuru, India, 10–11 December 2021; pp. 111–117. https://doi.org/10.1109/ICEECCOT52851.2021.9708041.
5. Dambra, S.; Han, Y.; Aonzo, S.; Kotzias, P.; Vitale, A.; Caballero, J.; Balzarotti, D.; Bilge, L. Decoding the Secrets of Machine Learning in Malware Classification: A Deep Dive into Datasets, Feature Extraction, and Model Performance. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*; Association for Computing Machinery: New York, NY, USA, 2023; pp. 589–603. https://doi.org/10.1145/3576915.3616589.
6. Pallavi, P.; Joshi, S.; Bairwa, A.K. Real-Time Detection of Polymorphic Malware Using AI and Synthetic Data Augmentation. In Proceedings of the 2025 XXX International Conference on Information, Communication and Automation Technologies (ICAT), Sarajevo, Bosnia and Herzegovina, 2–4 September 2025; pp. 1–6. https://doi.org/10.1109/ICAT66432.2025.11189272.
7. Abdrakhmanov, B. Hybrid AI-Based Static Analysis for Malware Detection: A Feature Engineering and Model Optimization Approach. In Proceedings of the 2025 IEEE 5th International Conference on Smart Information Systems and Technologies (SIST), Astana, Kazakhstan, 14–16 May 2025; pp. 1–6. https://doi.org/10.1109/SIST61657.2025.11139269.
8. Manikandan, K.P.; Reddy Onteddu, N.; Chilimi, A.K. Next-Gen Malware Detection using AI: AI-Powered Malware Threat Detection and Automated Malware Classification Through Machine Learning. In Proceedings of the 2025 International Conference on Intelligent Computing and Control Systems (ICICCS), Erode, India, 19–21 March 2025; pp. 581–588. https://doi.org/10.1109/ICICCS65191.2025.10985389.
9. Pradhan, U.; M, N.K.; Aditya, M.N. Malware Analysis Using Hashing and Explainable AI: A Comparative Study of LIME and SHAP Techniques. In Proceedings of the 2025 1st International Conference on AIML-Applications for Engineering & Technology (ICAET), Pune, India, 16–17 January 2025; pp. 1–6. https://doi.org/10.1109/ICAET63349.2025.10932147.
10. Prity, F.S.; Islam, M.S.; Fahim, E.H.; Hossain, M.M.; Bhuiyan, S.H.; Islam, M.A.; Raquib, M. Machine learning-based cyber threat detection: An approach to malware detection and security with explainable AI insights. *Hum.-Intell. Syst. Integr.* **2024**, *6*, 61–90. https://doi.org/10.1007/s42454-024-00055-7.
11. Charmet, F.; Tanuwidjaja, H.C.; Ayoubi, S.; Gimenez, P.F.; Han, Y.; Jmila, H.; Blanc, G.; Takahashi, T.; Zhang, Z. Explainable artificial intelligence for cybersecurity: A literature survey. *Ann. Telecommun.* **2022**, *77*, 789–812. https://doi.org/10.1007/s12243-022-00926-7.
12. Ofusori, L.; Bokaba, T.; Mhlongo, S. Explainability and interpretability of artificial intelligence use in cybersecurity. *Discov. Comput.* **2025**, *28*, 241. https://doi.org/10.1007/s10791-025-09760-6.
13. Mohamed, N. Artificial intelligence and machine learning in cybersecurity: A deep dive into state-of-the-art techniques and future paradigms. *Knowl. Inf. Syst.* **2025**, *67*, 6969–7055. https://doi.org/10.1007/s10115-025-02429-y.
14. Andresini, G.; Appice, A.; Belvedere, V.; Fiameni, G.; Malerba, D. Anakin: Explainable android malware detection with graph neural networks. *Cybersecurity* **2026**, *9*, 116. https://doi.org/10.1186/s42400-026-00552-z.
15. Zhang, S.H.; Kuo, C.C.; Yang, C.S. Static PE Malware Type Classification Using Machine Learning Techniques. In Proceedings of the 2019 International Conference on Intelligent Computing and its Emerging Applications (ICEA), Tainan, Taiwan, 30 August–1 September 2019; pp. 81–86. https://doi.org/10.1109/ICEA.2019.8858297.
16. Irawan, C.; Mantoro, T.; Ayu, M.A. Malware Detection and Classification Model Using Machine Learning Random Forest Approach. In Proceedings of the 2021 IEEE 7th International Conference on Computing, Engineering and Design (ICCED), Sukabumi, Indonesia, 5–6 August 2021; pp. 1–5. https://doi.org/10.1109/ICCED53389.2021.9664858.
17. Bruzzese, R. Building Visual Malware Dataset using VirusShare Data and Comparing Machine Learning Baseline Model to CoAtNet for Malware Classification. In *Proceedings of the 16th International Conference on Machine Learning and Computing*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 735–741. https://doi.org/10.1145/3651671.3651735.
18. Gaol, D.C.L.; Hendrawan.; Ismail, S.J.I. Development of Malware Classification Technique Based on Machine Learning Neural Network and High Order Markov Chain. In Proceedings of the 2022 8th International Conference on Wireless and Telematics (ICWT), Yogyakarta, Indonesia, 21–22 July 2022; pp. 1–6. https://doi.org/10.1109/ICWT55831.2022.9935479.

19. Trizna, D. Quo Vadis: Hybrid Machine Learning Meta-Model Based on Contextual and Behavioral Malware Representations. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 726–735. https://doi.org/10.1145/3560830.3563726.
20. Rana, D.S.; Dimri, S.C. Machine Learning Enables Malware Detection and Classification Techniques. In Proceedings of the 2024 IEEE International Conference on Computing, Power and Communication Technologies (IC2PCT), Greater Noida, India, 9–10 February 2024; pp. 1215–1221. https://doi.org/10.1109/IC2PCT60090.2024.10486211.
21. Yousuf, M.I.; Anwer, I.; Shakir, T.; Siddiqui, M.; Shahid, M. Multi-feature Dataset for Windows PE Malware Classification. *arXiv* **2022**, arXiv:2210.16285.
22. Syeda, D.Z.; Asghar, M.N. Dynamic malware classification and api categorisation of windows portable executable files using machine learning. *Appl. Sci.* **2024**, *14*, 1015.
23. Nagasivam, V. Malware Classification and Labelling using Deep Neural Networks. 2020. Available online: https://github.com/VISWESWARAN1998/Malware-Classification-and-Labelling (accessed on 17 April 2026).
24. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware images: Visualization and automatic classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security*; Association for Computing Machinery: New York, NY, USA, 2011; pp. 1–7.
25. Tekaya, Mohamed Taoufik. Malware Detection Using Machine Learning (MDML), 2022. GitHub Repository. Available online: https://github.com/MohamedTaoufik/MDML/ (accessed on 5 February 2026).
26. Center for Cybersecurity and Critical Infrastructure Protection, Kadir Has University. API Sequences Malware Datasets, 2021. GitHub Repository. Available online: https://github.com/khas-ccip/api_sequences_malware_datasets (accessed on 5 February 2026).
27. Yang, L.; Ciptadi, A.; Laziuk, I.; Ahmadzadeh, A.; Wang, G. BODMAS: An open dataset for learning based temporal analysis of PE malware. In *Proceedings of the 2021 IEEE Security and Privacy Workshops (SPW)*; IEEE: New York, NY, USA, 2021; pp. 78–84.
28. UCI Machine Learning Repository. Malware static and dynamic features VxHeaven and Virus Total. 2019. Available online: https://doi.org/10.24432/C58K6H (accessed on 12 April 2026).
29. Joyce, R.J.; Amlani, D.; Nicholas, C.; Raff, E. MOTIF: A Malware Reference Dataset with Ground Truth Family Labels. *Comput. Secur.* **2023**, *124*, 102921.
30. Joe Beach Capital. Windows Malwares. Kaggle Dataset. n.d. Available online: https://www.kaggle.com/datasets/joebeachcapital/windows-malwares (accessed on 12 April 2026).
31. Ikram, B.A.O.; Bouhorma, M.; Boudhir Anouar, A.; El Aachak, L.; Zafar, B. Machine Learning Application for Malwares Classification Using Visualization Technique. In *Proceedings of the 4th International Conference on Smart City Applications*; Association for Computing Machinery: New York, NY, USA, 2019; pp. 1–8. https://doi.org/10.1145/3368756.3369098.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.